KatanaKIT CSS — Documentation

A lightweight, modular SCSS mini-framework — design tokens, utility classes and layout mixins with zero runtime overhead.

Table of Contents

Start Here

KatanaKIT CSS

Why KatanaKIT CSS

KatanaKIT CSS is a **pure SCSS** framework inspired by the utility-first approach of Tailwind CSS, but without a build pipeline you don't control. Everything is generated at compile time from `!default` design-token maps:

- **Design tokens** — fonts, spacing, radius, shadows, z-layers, durations and easings as SCSS maps, emitted as CSS custom properties.
- **Color system** — 6 palettes × 7 shades + special colors, with utility classes, hover variants and an automatic dark theme.
- **Layout mixins** — breakpoints (7 tiers, mobile-first), a full CSS Grid toolkit and flexbox helpers.
- **Utility classes** — spacing with directional padding and margin (`.p-*`, `.px-*`, `.py-*`, `.pt-*` … `.m-*`, `.mx-*`, `.my-*` …), `gap-*`/`gap-x-*`/`gap-y-*`, text sizes (`.text-xs` … `.text-4xl`) and border widths (`.border`, `.border-0`/2/4/8), plus sizing, flex, effects, typography and layout generated from `!default` maps.
- **Extended preflight** — a Tailwind-style reset that normalises typography, lists, links, forms and media on top of token-driven defaults.
- **`@apply`-style composition** — a tiny registry system so you can compose utilities inside your components.

Quick install

```
npm install katanakit-css
```

```scss
// Full stylesheet (reset + tokens + utilities)
@use "katanakit-css/src/scss/main";

// Or compose your own from modules
@use "katanakit-css/src/scss/functions" as f;
@use "katanakit-css/src/scss/variables" as v;
@use "katanakit-css/src/scss/mixins" as m;
```

See Getting Started for the full guide and the API Reference for every function, mixin and map.

Playground

The repository ships with a Vite demo (`yarn dev` → http://localhost:4321) with a **version switcher** button: it toggles between the live SCSS (HMR) and published framework builds compiled by `yarn build:versions` into `public/versions/`.

Getting Started

This guide walks you through installing `katanakit-css` and writing your first styles. It assumes basic familiarity with Sass module syntax (`@use`). If you are new to the framework, skim the Introduction first — this guide goes a bit deeper on concrete setup options.

The full, code-verified API surface lives in the API Reference.

Prerequisites

- **Node.js** (>= 20 is a safe floor) to install the package.
- A **Sass compiler** (Dart Sass is what this project is tested against) when you consume the SCSS source. `sass` is a devDependency of the repo, but it is your responsibility in downstream projects.
- The **precompiled stylesheet** (`dist/css/katanakit.css`) needs no tooling.

There is **no runtime**: `katanakit-css` is pure build-time SCSS/CSS.

Installation

```
npm install katanakit-css
# or
yarn add katanakit-css
```

Choosing an entry point

Entry	What you get
`dist/css/katanakit.css`	Compiled, minified **full sheet** (reset + tokens + utilities).
`katanakit-css/src/scss/main`	Full sheet as SCSS you compile yourself (adds browser prefixes, custom purging, tree-shaking of SCSS modules).
`katanakit-css/src/`	Individual modules you compose into a custom

Entry	What you get
`scss/*`	sheet.

All SCSS imports are **lowercase `@use`** with an explicit namespace. There is no legacy `@import` anywhere.

Path 1 — precompiled CSS

Link it or import it:

```
<link rel="stylesheet" href="/node_modules/dist/css/katanakit.css" />
```

Or import it from your JavaScript entry in a bundler-based app:

```
import "katanakit-css/dist/css/katanakit.css";
```

The sheet already contains every utility class the framework can generate, so you can start writing markup immediately:

```
<body class="bg-neutral-100 text-neutral-500">
  <main class="p-8">
    <h1 class="text-2xl mb-4">Hello KatanaKIT</h1>
    <div class="card"><!-- styled by your own component CSS --></div>
  </main>
</body>
```

Trade-off: the full sheet is convenient, but ships every utility. For smaller output, use Path 2 or 3 and let your build (or PurgeCSS, as the demo does) remove unused classes.

Path 2 — compile the full SCSS entry

```
// styles/main.scss
@use "katanakit-css/src/scss/main";
```

Add `styles/main.scss` to your Sass pipeline. This compiles the same contents as the precompiled sheet: a Tailwind-style preflight reset, token custom properties in `:root`, color custom properties and utilities, and all utility generators.

To use the `@apply` registry or the mixins in your own SCSS, load the partials too:

```scss
@use "katanakit-css/src/scss/mixins" as m;

.card {
  @include m.apply(flex, flex-col, p-4, rounded-lg, shadow-md, bg-
white);

  @include m.md {
    @include m.apply(flex-row, gap-6);
  }
}
```

Path 3 — compose a custom sheet

You only pay for the modules you load. Every partial is importable; all token maps are `!default`.

```scss
// styles/theme.scss
@use "katanakit-css/src/scss/reset";
@use "katanakit-css/src/scss/variables" as v;
@use "katanakit-css/src/scss/functions" as f;
@use "katanakit-css/src/scss/mixins" as m;
@use "katanakit-css/src/scss/utilities" as u;

// Tokens and color variables
@include v.generate-css-tokens();
@include v.generate-css-vars();

// Color utilities: text, backgrounds and hover text/bg
@include v.all-utilities();

// Utility generators (opt-in)
@include u.generate-all-utilities();
```

Remember: just loading the `utilities` module automatically emits spacing classes — padding and margin in every direction (`.p-*`, `.px-*`, `.py-*`, `.pt-*`, `.m-*`, `.mx-*`, `.my-*`, …) plus `gap-*`/`gap-x-*`/`gap-y-*` — text sizes (`.text-xs` … `.text-4xl`) and border widths (`.border`, `.border-0`/2/4/8), as well as `.gap`, `.shadow` and `.rounded`. `generate-all-utilities()` adds sizing, flex, effects and layout classes. Skip the generators you do not need.

Overriding design tokens

Token maps are `!default`, so you override them with a Sass `with` clause. Because a module can only be configured once per compilation, configure `variables` **before** anything else loads it:

```scss
// styles/tokens.scss — configure first
@use "katanakit-css/src/scss/variables" with (
  $font-families: (
    "sans-serif": (system-ui, -apple-system, "Segoe UI", sans-serif),
    "serif": (Georgia, "Times New Roman", serif),
    "mono": (ui-monospace, "SFMono-Regular", Menlo, monospace),
  ),
  $spacing-scale: (
    "0": 0, "1": 0.25rem, "2": 0.5rem, "3": 0.75rem, "4": 1rem,
    "6": 1.5rem, "8": 2rem, "12": 3rem, "16": 4rem, "20": 5rem,
  ),
  $breakpoint-sizes: ("xs": 0, "sm": 600px, "md": 900px, "lg": 1200px,
"xl": 1440px, "2xl": 1600px, "3xl": 1920px),
);

// styles/main.scss — then consume
@use "tokens";
@use "katanakit-css/src/scss/main";
```

Utility maps (`_utilities.scss`) are configured the same way if you load them directly, e.g. `$sizing-map`, `$spacing-map`, `$radius-map`, `$shadow-map`, `$z-layers-map`, `$opacity-values`, `$font-weight-values`, etc.

Colors and dark theme

```scss
@use "katanakit-css/src/scss/variables" as v;

@include v.generate-css-vars();
@include v.theme("dark"); // palettes inverted under :root[data-theme="dark"]
```

```html
<html data-theme="dark">
  <body class="bg-neutral-100 text-neutral-500">
    <!-- neutral 100 is now the darkest tone when the dark theme is active -->
```

```
  </body>
</html>
```

Functions give you the raw colors at build time: `v.get-color("info", 300)`, `v.get("success")`, `v.alpha("danger", 500, 0.4)`.

Where to go next

- API Reference — complete list of functions, mixins and maps per module.
- Architecture — module graph and build flow.
- Roadmap — shipped and planned work.
- Demo — run `yarn dev` and inspect `index.html` + `demo/main.js` for a working example of everything above.

Core Concepts

Design Tokens

Every design decision in KatanaKIT CSS lives in a set of `!default` SCSS maps under `src/scss/_variables.scss`. Because the maps are `!default`, you override them **once per compilation** with a Sass `with` clause, and both the token accessors and the generated CSS update accordingly.

All token maps are consumed through the `variables` module:

```
@use "katanakit-css/src/scss/variables" as v;
```

Token maps

Map	Keys	Feeds
`$font-families`	`sans-serif`, `serif`, `mono`	`v.font-family()`
`$shadow-sizes`	`default`, `sm`, `md`, `lg`, `xl`, `2xl`, `inner`	`v.shadow()`
`$container-sizes`	`sm`, `md`, `lg`, `xl`, `2xl`, `3xl`, `4xl`, `5xl`, `6xl`	`v.container()`
`$breakpoint-sizes`	`xs`, `sm`, `md`, `lg`, `xl`, `2xl`, `3xl`	`v.breakpoint()`
`$spacing-scale`	`0`, `1`, `2`, `3`, `4`, `5`, `6`, `8`, `10`, `12`, `16`, `20`, `24`, `32`	`v.spacing()`
`$spacing-map`	full scale incl. fractions	`.p-*` `.m-*` `.gap-*` classes

Map	Keys	Feeds
	(`0`5, `1-5`, `2-5`, `3-5`, `base`, 9, 11, 14, … 96)	
`$radius`	`default`, `sm`, `md`, `lg`, `xl`, `2xl`, `3xl`, `full`	`v.radius()`
`$z-layers`	`auto`, `0`, `10`, `20`, `30`, `40`, `50`, `dropdown`, `sticky`, `fixed`, `modal`, `popover`, `tooltip`	`v.z()`
`$transition-durations`	`75`, `100`, `150`, `200`, `300`, `500`, `700`, `1000`	`v.duration()`
`$transition-easings`	`linear`, `in`, `out`, `in-out`	`v.ease()`

Two spacing maps coexist on purpose:

- `$spacing-scale` is the **semantic scale** used by the `v.spacing()` accessor and exposed as `--spacing-*` custom properties.
- `$spacing-map` is the **class scale** — the single source of truth for the `.p-*`, `.m-*` and `.gap-*` utility classes. It adds fractional steps (`0`5, `1-5`, `2-5`, `3-5`), `base` (1px) and the intermediate steps (7, 9, 11, 14, 28, … 96) that the class generators emit.

Accessor functions

Every accessor validates its key at compile time and raises an `@error` that lists the valid keys when you miss.

Signature	Returns	Example
`v.font-family($name)`	font stack list	`v.font-family("sans-serif")`
`v.shadow($size: "default")`	shadow list	`v.shadow("md")`
`v.container($size)`	length	`v.container("lg")` → `32rem`
`v.breakpoint($name)`	px length	`v.breakpoint("md")` → `768px`
`v.spacing($size)`	length (number or string key)	`v.spacing(4)` → `1rem`
`v.radius($size: "default")`	length	`v.radius("xl")` → `0.75rem`

Signature	Returns	Example
`v.z($layer)`	number or `auto`	`v.z("modal")` → `1040`
`v.duration($ms)`	duration	`v.duration(200)` → `200ms`
`v.ease($name: "in-out")`	timing function	`v.ease("out")`

```scss
@use "katanakit-css/src/scss/variables" as v;

.card {
  padding: v.spacing(4);
  border-radius: v.radius("default");
  box-shadow: v.shadow("lg");
  font-family: v.font-family("sans-serif");
}

.overlay {
  z-index: v.z("modal");
  transition: opacity v.duration(200) v.ease("out");
}
```

Generating CSS custom properties

`generate-css-tokens()` emits the token families as custom properties in `:root`. Pass `false` to skip a family:

```scss
@mixin generate-css-tokens(
  $fonts: true, $shadows: true, $containers: true, $spacing: true,
  $radius-tokens: true, $z: true, $durations: true, $easings: true
)

@use "katanakit-css/src/scss/variables" as v;

@include v.generate-css-tokens();

// Only fonts and shadows:
@include v.generate-css-tokens($containers: false, $spacing: false);
```

Compiled output:

```scss
:root {
  --font-sans-serif: ui-sans-serif, system-ui, -apple-system, …;
  --shadow-md: 0 4px 6px -1px rgb(0 0 0 / 0.1), 0 2px 4px -2px rgb(0 0
0 / 0.1);
```

```
  --container-xl: 36rem;
  --spacing-4: 1rem;
  --radius-full: 9999px;
  --z-tooltip: 1060;
  --duration-200: 200ms;
  --ease-in-out: cubic-bezier(0.4, 0, 0.2, 1);
}
```

The variable names carry no brand prefix: `--font-*`, `--shadow-*`, `--container-*`, `--spacing-*`, `--radius-*`, `--z-*`, `--duration-*` and `--ease-*`.

Overriding tokens

Configure the module with `with` **before** anything else loads it — a module can only be configured once per compilation:

```scss
// styles/tokens.scss — configure first
@use "katanakit-css/src/scss/variables" as v with (
  $spacing-scale: (
    "0": 0,
    "1": 0.5rem,
    "2": 1rem,
    "3": 1.5rem,
    "4": 2rem
  ),
  $radius: (
    "default": 0.5rem,
    "full": 9999px
  ),
  $breakpoint-sizes: (
    "xs": 0,
    "sm": 600px,
    "md": 900px,
    "lg": 1200px,
    "xl": 1440px
  )
);

// styles/main.scss — then consume
@use "tokens";
@use "katanakit-css/src/scss/main";
```

The accessors, the `--spacing-*` custom properties and the utility classes that derive from the configured maps all pick up the new values.

Related pages

- Colors — the color palettes and their accessors.
- Breakpoints — responsive breakpoint tiers.
- Functions — pure helper functions.

Colors

KatanaKIT CSS ships **6 palettes with 7 shades each** (`100` lightest → `700` darkest) plus four special colors. Everything lives in the `variables` module and is emitted as utility classes, CSS custom properties and hover variants.

```scss
@use "katanakit-css/src/scss/variables" as v;
```

Palettes

Each palette follows the same lightness ramp: `100`–`300` are background tones, `400` is the vivid brand tone, `500`–`700` are text tones.

Neutral

Purple

Info

Warning

Danger

Success

Special colors

`current` is also a special color (`currentColor`), but it has no visible swatch of its own — it resolves to the element's `color` value.

Tone values

Stored as `hsl(hue, saturation%, lightness%)`:

Palette	100	200	300	400	500	600	700
`neutral`	0 0% 93%	0 0% 86%	0 0% 71%	0 0% 50%	0 0% 35%	0 0% 24%	0 0% 12%
`purple`	269 60% 93%	269 70% 86%	269 70% 71%	269 66% 50%	269 66% 35%	269 60% 24%	269 50% 12%
`info`	215 95% 93%	215 95% 86%	215 95% 71%	215 95% 50%	215 95% 35%	215 95% 24%	215 95% 12%

Palette	100	200	300	400	500	600	700
`warning`	49 100% 93%	49 100% 86%	49 100% 71%	49 100% 50%	49 100% 35%	49 90% 24%	49 70% 12%
`danger`	3 95% 93%	3 95% 86%	3 95% 71%	3 95% 50%	3 95% 35%	3 95% 24%	3 95% 12%
`success`	147 95% 93%	147 95% 86%	147 95% 71%	147 95% 50%	147 95% 35%	147 95% 24%	147 95% 12%

Special colors: `black` `hsl(0, 0%, 3%)` · `white` `hsl(0, 0%, 98%)` · `transparent` `transparent` · `current` `currentColor`.

Accessor functions

Signature	Returns
`v.get-color($name, $shade: 300, $alpha: 1)`	A palette shade (or a special color when `$name` is special). Applies alpha when `$alpha != 1`.
`v.get($name, $alpha: 1)`	Shorthand: `get-color($name, 500, $alpha)`.
`v.alpha($name, $shade: 500, $alpha: 0.5)`	Shorthand for transparent variants.

```scss
@use "katanakit-css/src/scss/variables" as v;

color: v.get-color("neutral", 500);  // hsl(0, 0%, 35%)
color: v.get("info");                // shade 500: hsl(215, 95%, 35%)
border: 1px solid v.alpha("warning", 400, 0.25);
background: v.alpha("purple", 100, 0.4);
```

Invalid palette or shade names raise a compile-time `@error`.

Generating CSS variables

`generate-css-vars()` emits one custom property per shade plus the specials:

```scss
@use "katanakit-css/src/scss/variables" as v;

@include v.generate-css-vars();
```

```scss
:root {
  --neutral-100: hsl(0, 0%, 93%);
  /* … every palette and shade … */
  --info-500: hsl(215, 95%, 35%);
```

```
  --black: hsl(0, 0%, 3%);
  --white: hsl(0, 0%, 98%);
  --transparent: transparent;
  --current: currentColor;
}

// Only the info palette, no specials:
@include v.generate-css-vars($palettes: "info", $include-special:
false);

// A custom root selector:
@include v.generate-css-vars($root: ".theme-brand");
```

`$palettes` accepts `null` (all), a name, a list of names, or a map emitted as-is — the mechanism the dark theme uses to publish inverted palettes.

Utility classes

The color mixins generate the classes you use in markup:

Mixin	Classes
`v.text-utilities()`	`.text-{palette}-{100…700}` + `.text-black` `.text-white` `.text-transparent` `.text-current`
`v.bg-utilities()`	`.bg-{palette}-{100…700}` + `.bg-black` `.bg-white` `.bg-transparent` `.bg-current`
`v.border-utilities()`	`.border-{palette}-{100…700}` + the four specials
`v.hover-utilities()`	`.hover-text-*:hover` and `.hover-bg-*:hover`
`v.all-utilities()`	All four mixins at once (called by `main.scss`).

See Text Color, Background Color and Border Color for the complete class tables.

Breakpoints

Breakpoints drive the responsive mixins. The tiers live in two places that must stay in sync: `v.$breakpoint-sizes` (token accessor) and `m.$breakpoints` (mixin engine). Both are `!default` maps with the same defaults.

Tier	Width
`xs`	`0`
`sm`	`640px`
`md`	`768px`
`lg`	`1024px`
`xl`	`1280px`
`2xl`	`1536px`
`3xl`	`1920px`

```scss
@use "katanakit-css/src/scss/mixins" as m;
@use "katanakit-css/src/scss/variables" as v;

// The token accessor returns raw values for your own media queries:
$tablet: v.breakpoint("md"); // 768px
```

The keys containing digits (`2xl`, `3xl`) are first-class names — always pass them as strings.

The generic mixin

`breakpoint($from, $direction: up, $to: null)` opens a `@media` query. `$from` (and `$to`) accept a tier name or a raw px number. `bp()` is an alias that forwards its arguments.

Direction	Media query	Requires `$to`
`up`	`(min-width: X)`	no
`down`	`(max-width: calc(X - 0.02px))`	no
`only`	`(min-width: X) and (max-width: calc(Y - 0.02px))`	yes
`between`	`(min-width: X) and (max-width: Y)`	yes

```scss
@use "katanakit-css/src/scss/mixins" as m;

@include m.breakpoint("md", up) {
  .card { display: grid; }
}

@include m.bp("lg", down) { /* alias, same as breakpoint("lg", down) */ }
```

```scss
@include m.breakpoint("sm", only, "md") { /* tablets only */ }

@include m.breakpoint(900px, between, 1200px) { /* raw values work too
*/ }
```

Direction mixins (mobile-first)

Shortcut mixins for the `up` direction, including `xxl` / `xxxl` aliases for the digit keys:

```scss
@include m.xs   { /* ≥ 0      */ }
@include m.sm   { /* ≥ 640px  */ }
@include m.md   { /* ≥ 768px  */ }
@include m.lg   { /* ≥ 1024px */ }
@include m.xl   { /* ≥ 1280px */ }
@include m.xxl  { /* ≥ 1536px (2xl) */ }
@include m.xxxl { /* ≥ 1920px (3xl) */ }
```

Down mixins (desktop-first)

```scss
@include m.xs-down   { /* ≤ 0 */ }
@include m.sm-down   { /* ≤ 639.98px */ }
@include m.md-down   { /* ≤ 767.98px */ }
@include m.lg-down   { /* ≤ 1023.98px */ }
@include m.xl-down   { /* ≤ 1279.98px */ }
@include m.x2l-down  { /* ≤ 1535.98px (2xl) */ }
@include m.x3l-down  { /* ≤ 1919.98px (3xl) */ }

// Canonical aliases for the digit keys:
@include m.xxl-down  { /* = x2l-down */ }
@include m.xxxl-down { /* = x3l-down */ }
```

Combining both directions

A typical responsive card grid, mobile-first then constrained at the top end:

```scss
@use "katanakit-css/src/scss/mixins" as m;

.grid {
  display: grid;
  grid-template-columns: 1fr;

  @include m.sm { grid-template-columns: repeat(2, 1fr); }
  @include m.lg { grid-template-columns: repeat(4, 1fr); }
}
```

See the Breakpoints mixin reference for the complete technical reference of the generic mixin.

Feature queries

Beyond widths, the module ships media-feature shortcuts:

Mixin	Media query
`m.portrait`	`(orientation: portrait)`
`m.landscape`	`(orientation: landscape)`
`m.reduced-motion`	`(prefers-reduced-motion: reduce)`
`m.hoverable`	`(hover: hover) and (pointer: fine)`
`m.touch`	`(hover: none) and (pointer: coarse)`
`m.dark-mode`	`(prefers-color-scheme: dark)`
`m.light-mode`	`(prefers-color-scheme: light)`

```scss
@use "katanakit-css/src/scss/mixins" as m;

.hero-animation {
  animation: slide-up 300ms ease;

  @include m.reduced-motion {
    animation: none;
  }

  @include m.touch {
    // Larger hit targets on touch devices
    padding: 1.5rem;
  }
}
```

`m.dark-mode` and `m.light-mode` follow the **system** preference. For the class-based theme, see Dark Mode.

Overriding the tiers

Because both maps are `!default`, configure them before first use:

```scss
@use "katanakit-css/src/scss/variables" as v with (
  $breakpoint-sizes: (
    "xs": 0,
    "sm": 600px,
    "md": 900px,
    "lg": 1200px,
    "xl": 1440px,
    "2xl": 1600px
  )
);
@use "katanakit-css/src/scss/mixins" as m with (
  $breakpoints: (
    "xs": 0,
    "sm": 600px,
    "md": 900px,
    "lg": 1200px,
    "xl": 1440px,
    "2xl": 1600px
  )
);
```

Dark Mode

KatanaKIT CSS generates a class-based dark theme for free: `v.theme("dark")` takes the six palettes, **inverts each shade** (`100 ↔ 700`, `200 ↔ 600`, `300 ↔ 500`), and publishes the result as custom properties under `:root[data-theme="dark"]`.

The inversion applies to the **custom properties** (`--neutral-100`, …). The color utility classes (`.bg-neutral-100`, `.text-neutral-500`, …) compile to literal values and are theme-independent — use the variables in your own rules when you need theme-aware styles.

Enabling the dark theme

```scss
@use "katanakit-css/src/scss/variables" as v;

@include v.generate-css-vars(); // light defaults on :root
@include v.theme("dark");       // inverted palettes on :root[data-theme="dark"]

:root[data-theme="dark"] {
  --neutral-100: hsl(0, 0%, 12%);   /* was 700 */
  --neutral-200: hsl(0, 0%, 24%);   /* was 600 */
  /* … */
```

```
  --neutral-700: hsl(0, 0%, 93%);   /* was 100 */
}
```

Then toggle the theme by setting the attribute:

```
<html data-theme="dark">
  <body class="surface text-body">
    <!-- .surface/.text-body are YOUR rules that consume
         var(--neutral-100) / var(--neutral-500): under the
         dark theme those variables now hold the inverted tones -->
  </body>
</html>
```

```
// Flip between light and dark
document.documentElement.setAttribute("data-theme", "dark");
document.documentElement.removeAttribute("data-theme");
```

How the inversion works

The inversion is a pure map transformation: `$inverted: 800 - $shade`.

Light shade	Dark shade
100	700
200	600
300	500
400	400
500	300
600	200
700	100

This is why the tonal vocabulary stays stable: `bg-neutral-100` is always “the surface”, `text-neutral-500` is always “the readable text” — the actual HSL values swap underneath.

Building on the variables

The dark-theme variables are ordinary custom properties, so compose freely:

```
@use "katanakit-css/src/scss/variables" as v;

@include v.generate-css-vars();
@include v.theme("dark");
```

```
body {
  background-color: var(--neutral-100);
  color: var(--neutral-500);
}

.brand {
  color: var(--purple-400);
}
```

Note the color **utility classes** (`v.text-utilities()` etc.) compile to literal values, not `var()` references. If you need theme-aware classes, use the custom properties in your own rules, as above.

Overriding the dark palette

`theme()` accepts an `$overrides` map merged over the inverted result:

```
@include v.theme(
  "dark",
  (
    "neutral": (
      100: hsl(0, 0%, 8%),
      500: hsl(0, 0%, 88%)
    )
  )
);
```

Combining with the system preference

Pair the attribute-based theme with the `m.dark-mode` feature query to sync `color-scheme` (scrollbars, form controls) with the system preference:

```
@use "katanakit-css/src/scss/variables" as v;
@use "katanakit-css/src/scss/mixins" as m;

@include v.generate-css-vars();
@include v.theme("dark");

@include m.dark-mode {
  :root {
    color-scheme: dark;
  }
}
```

To follow the system preference automatically, set `data-theme` from JavaScript:

```
const mq = window.matchMedia("(prefers-color-scheme: dark)");
const apply = () =>
  document.documentElement.toggleAttribute("data-theme", mq.matches);

apply();
mq.addEventListener("change", apply);
```

Related pages

- Colors — palettes and `generate-css-vars()`.
- Breakpoints — the `m.dark-mode` feature query.
- Text Color and Background Color — the color utility classes.

@apply

KatanaKIT CSS ships a small `@apply`-style system so you can compose utilities inside your components **without** re-implementing them:

```
@use "katanakit-css/src/scss/mixins" as m;

.card {
  @include m.apply(flex, flex-col, p-4, rounded-lg, shadow-md, bg-
white);
}
```

Two mixins power the system:

Mixin	Behaviour
`m.register-utility($name, $styles)`	Registers a named utility. `$styles` is a map of CSS declarations.
`m.apply($utilities...)`	Emits every registered declaration for each name, in order. Throws an `@error` naming the offending utility when one is not registered.

Basic usage

```
@use "katanakit-css/src/scss/mixins" as m;

.card {
  @include m.apply(flex, flex-col, p-4, rounded-lg, shadow-md, bg-
white);
```

```
  &:hover {
    @include m.apply(shadow-lg);
  }
}

.button {
  @include m.apply(
    inline-flex,
    items-center,
    justify-center,
    px-4,
    py-2,
    rounded-md,
    font-semibold,
    transition-colors
  );

  &:hover {
    @include m.apply(bg-white, text-black);
  }
}
```

Compiles to plain declarations — no runtime, no duplicate class output:

```
.card {
  display: flex;
  flex-direction: column;
  padding: 1rem;
  border-radius: 0.5rem;
  box-shadow: 0 4px 6px -1px rgb(0 0 0 / 0.1), 0 2px 4px -2px rgb(0 0
0 / 0.1);
  background-color: hsl(0, 0%, 98%);
}
```

Registering your own utilities

The registry is open — register project-specific utilities before using them:

```
@use "katanakit-css/src/scss/mixins" as m;

@include m.register-utility("fade-in", (animation: fade-in 300ms
ease));
```

```
.hero {
  @include m.apply(fade-in, p-8, text-2xl);
}
```

Automatically registered utilities

Loading the `mixins` module registers a large set of names — most of them derived from the **same maps** that drive the class generators, so names match the class names exactly (`p-4`, `m-2`, `gap-4`, `rounded-lg`, `shadow-md`, `z-10`, `duration-200`, `ease-out`, `opacity-50`, …).

Category	Registered names
Display	`block`, `inline-block`, `inline`, `flex`, `inline-flex`, `grid`, `hidden`, `table`, `table-row`, `table-cell`, `contents`, `inline-grid`
Flexbox	`flex-row`, `flex-row-reverse`, `flex-col`, `flex-col-reverse`, `flex-wrap`, `flex-nowrap`, `items-start`, `items-center`, `items-end`, `items-stretch`, `justify-start`, `justify-center`, `justify-end`, `justify-between`, `justify-around`, `justify-evenly`
Spacing	for every key of `$spacing-map`: `p-*` `px-*` `py-*` `pt-*` `pr-*` `pb-*` `pl-*`, `m-*` `mx-*` `my-*` `mt-*` `mr-*` `mb-*` `ml-*`, `gap-*` `gap-x-*` `gap-y-*`
Typography	`text-xs` … `text-4xl`, `text-left`, `text-center`, `text-right`, `font-thin` … `font-black`
Sizing	`w-full`, `w-screen`, `w-auto`, `h-full`, `h-screen`, `h-auto`
Borders	`border`, `border-0`, `border-2`, `border-4`, `border-8`, `rounded-none` … `rounded-full`, `rounded`
Effects	`shadow-none` … `shadow-2xl` + `shadow` + `shadow-inner`, `opacity-0` … `opacity-100`, `z-auto` … `z-tooltip`, `duration-75` … `duration-1000`, `ease-`

Category	Registered names
	`linear` … `ease-in-out`
Position	`static`, `fixed`, `absolute`, `relative`, `sticky`
Overflow	`overflow-auto`, `overflow-hidden`, `overflow-scroll`, `overflow-visible`
Colors	`text-white`, `text-black`, `bg-white`, `bg-black`, `bg-transparent`
Whitespace	`whitespace-nowrap`, `whitespace-pre`, `whitespace-normal`, `wrap-break-word`

Registry-only utilities

Important. Registering a utility for `apply` does **not** create a CSS class. A small set of names exists only inside the registry — you can use them with `apply()`, but they will never appear as classes in the compiled CSS:

- `flex-1`, `flex-auto`, `flex-none`
- `grid-cols-1`, `grid-cols-2`, `grid-cols-3`, `grid-cols-4`, `grid-cols-6`, `grid-cols-12`
- `col-span-full`
- `transition`, `transition-colors`, `transition-opacity`, `transition-transform`

```scss
@use "katanakit-css/src/scss/mixins" as m;

.hero-grid {
  @include m.apply(grid, grid-cols-12, gap-4);

  .main { @include m.apply(col-span-full); }
  .aside { @include m.apply(flex-1); }
}
```

```html
<!-- Wrong: .flex-1 is NOT emitted as a class by the framework -->
<div class="flex-1">…</div>
```

Errors

Applying an unregistered name fails loudly at compile time:

```
@include m.apply(flex, flex-col, text-5xl);
// Error: apply(): utilidad 'text-5xl' no registrada.
//        Usa register-utility() primero.
```

Related pages

- Getting Started — composing sheets with modules.
- Examples — the button/card example in full.

Utilities

Border Width

Utilities for controlling the width of an element's borders. `.border` (1px solid) is emitted automatically with the `utilities` module; the other widths come from the `$border-width-map`.

Quick reference

Class	Properties
`.border`	`border-width: 1px; border-style: solid;`
`.border-0`	`border-width: 0;`
`.border-2`	`border-width: 2px;`
`.border-4`	`border-width: 4px;`
`.border-8`	`border-width: 8px;`

Basic usage

```
<div class="border border-purple-500 ...">border</div>
<div class="border-2 border-purple-500 ...">border-2</div>
<div class="border-4 border-purple-500 ...">border-4</div>
```

Widths are separate from colors: add a color from Border Color or the default (`currentColor`) applies.

Customizing

```
@use "katanakit-css/src/scss/utilities" as u with (
  $border-width-map: (
    "0": 0,
    "2": 2px,
    "4": 4px
```

```
  )
);
```

Display

Utilities for controlling the display box type of an element. Generated from the `$display-values` map (a map so `hidden` can map to `none`).

Quick reference

Class	Properties
`.block`	`display: block;`
`.inline-block`	`display: inline-block;`
`.inline`	`display: inline;`
`.flex`	`display: flex;`
`.inline-flex`	`display: inline-flex;`
`.grid`	`display: grid;`
`.inline-grid`	`display: inline-grid;`
`.table`	`display: table;`
`.table-row`	`display: table-row;`
`.table-cell`	`display: table-cell;`
`.hidden`	`display: none;`
`.contents`	`display: contents;`

Basic usage

Set the display type of an element:

```
<div class="block ...">block</div>
<div class="inline-block ...">inline-block</div>
<div class="inline ...">inline</div>
```

Using flex and grid

`flex` and `grid` create the containers that every flexbox and grid utility works on:

```
<div class="flex gap-2">…</div>
<div class="grid gap-2" style="grid-template-columns: repeat(3,
1fr)">…</div>
```

Hiding elements

Use `hidden` to remove an element from the document flow entirely:

```
<div class="flex gap-2">
  <div class="p-4 ...">visible</div>
  <div class="hidden p-4 ...">hidden</div>
</div>
```

Unlike `opacity-0`, `hidden` takes no space and receives no pointer events.

Using table displays

The `table-*` utilities let you apply table layout semantics to any structure:

```
<div class="table">
  <div class="table-row">
    <div class="table-cell p-2">Cell A</div>
    <div class="table-cell p-2">Cell B</div>
  </div>
</div>
```

Customizing

Override the `$display-values` map in the `mixins` module before generating layout utilities:

```
@use "katanakit-css/src/scss/mixins" as m with (
  $display-values: (
    block: block,
    flex: flex,
    grid: grid,
    hidden: none
  )
);
@use "katanakit-css/src/scss/utilities" as u;

@include u.generate-layout-utilities();
```

Flex Direction

Utilities for controlling the direction of flex items. Requires a flex container (`.flex` or `.inline-flex` — see Display).

Quick reference

Class	Properties
`.flex-row`	`flex-direction: row;`
`.flex-row-reverse`	`flex-direction: row-reverse;`
`.flex-col`	`flex-direction: column;`
`.flex-col-reverse`	`flex-direction: column-reverse;`

Basic usage

Lay items out in a row (the default) or a column:

```
<div class="flex flex-row gap-2">…</div>
<div class="flex flex-col gap-2">…</div>
```

Reversing the direction

`flex-row-reverse` and `flex-col-reverse` keep the same layout axis but flip the visual order of the items:

```
<div class="flex flex-row-reverse gap-2">…</div>
```

Customizing

Direction classes are generated from the `$flex-direction-map` in the `mixins` module:

```
@use "katanakit-css/src/scss/mixins" as m with (
  $flex-direction-map: (
    "row": row,
    "col": column
  )
);
@use "katanakit-css/src/scss/utilities" as u;

@include u.generate-flex-utilities();
```

Font Size

Utilities for controlling the font size of an element, generated from the `$font-size-map`. These classes are emitted automatically when the `utilities` module is loaded — no generator call needed.

Quick reference

Class	Properties
`.text-xs`	`font-size: 0.75rem;`
`.text-sm`	`font-size: 0.875rem;`
`.text-base`	`font-size: 1rem;`
`.text-lg`	`font-size: 1.125rem;`
`.text-xl`	`font-size: 1.25rem;`
`.text-2xl`	`font-size: 1.5rem;`
`.text-3xl`	`font-size: 1.875rem;`
`.text-4xl`	`font-size: 2.25rem;`

Basic usage

```
<p class="text-xs ...">The quick brown fox — text-xs</p>
<p class="text-sm ...">The quick brown fox — text-sm</p>
<!-- … up to text-4xl -->
```

Customizing

Override the `$font-size-map` in the `utilities` module before it loads:

```
@use "katanakit-css/src/scss/utilities" as u with (
  $font-size-map: (
    "sm": 0.75rem,
    "base": 1rem,
    "xl": 1.5rem
  )
);
```

Padding

Utilities for controlling an element's padding.

Quick reference

Every value of `$spacing-map` is emitted for the seven prefixes: `p-*` (all sides), `px-*`/`py-*` (horizontal/vertical) and `pt-*`/`pr-*`/`pb-*`/`pl-*` (one side).

Scale	`p-*`	`px-*`	`py-*`	`pt-*`	`pr-*`	`pb-*`	`pl-*`
`0` → `0`	`.p-0`	`.px-0`	`.py-0`	`.pt-0`	`.pr-0`	`.pb-0`	`.pl-0`
`05` → `0.125re`	`.p-05`	`.px-05`	`.py-05`	`.pt-05`	`.pr-05`	`.pb-05`	`.pl-05`

Scale	p-*	px-*	py-*	pt-*	pr-*	pb-*	pl-*
base → 1px	.p-base	.px-base	.py-base	.pt-base	.pr-base	.pb-base	.pl-base
1 → 0.25rem	.p-1	.px-1	.py-1	.pt-1	.pr-1	.pb-1	.pl-1
1-5 → 0.375rem	.p-1-5	.px-1-5	.py-1-5	.pt-1-5	.pr-1-5	.pb-1-5	.pl-1-5
2 → 0.5rem	.p-2	.px-2	.py-2	.pt-2	.pr-2	.pb-2	.pl-2
2-5 → 0.625rem	.p-2-5	.px-2-5	.py-2-5	.pt-2-5	.pr-2-5	.pb-2-5	.pl-2-5
3 → 0.75rem	.p-3	.px-3	.py-3	.pt-3	.pr-3	.pb-3	.pl-3
3-5 → 0.875rem	.p-3-5	.px-3-5	.py-3-5	.pt-3-5	.pr-3-5	.pb-3-5	.pl-3-5
4 → 1rem	.p-4	.px-4	.py-4	.pt-4	.pr-4	.pb-4	.pl-4
5 → 1.25rem	.p-5	.px-5	.py-5	.pt-5	.pr-5	.pb-5	.pl-5
6 → 1.5rem	.p-6	.px-6	.py-6	.pt-6	.pr-6	.pb-6	.pl-6
7 → 1.75rem	.p-7	.px-7	.py-7	.pt-7	.pr-7	.pb-7	.pl-7
8 → 2rem	.p-8	.px-8	.py-8	.pt-8	.pr-8	.pb-8	.pl-8
9 → 2.25rem	.p-9	.px-9	.py-9	.pt-9	.pr-9	.pb-9	.pl-9
10 → 2.5rem	.p-10	.px-10	.py-10	.pt-10	.pr-10	.pb-10	.pl-10
11 → 2.75rem	.p-11	.px-11	.py-11	.pt-11	.pr-11	.pb-11	.pl-11
12 → 3rem	.p-12	.px-12	.py-12	.pt-12	.pr-12	.pb-12	.pl-12

Scale	p-*	px-*	py-*	pt-*	pr-*	pb-*	pl-*
14 → 3.5rem	.p-14	.px-14	.py-14	.pt-14	.pr-14	.pb-14	.pl-14
16 → 4rem	.p-16	.px-16	.py-16	.pt-16	.pr-16	.pb-16	.pl-16
20 → 5rem	.p-20	.px-20	.py-20	.pt-20	.pr-20	.pb-20	.pl-20
24 → 6rem	.p-24	.px-24	.py-24	.pt-24	.pr-24	.pb-24	.pl-24
28 → 7rem	.p-28	.px-28	.py-28	.pt-28	.pr-28	.pb-28	.pl-28
32 → 8rem	.p-32	.px-32	.py-32	.pt-32	.pr-32	.pb-32	.pl-32
36 → 9rem	.p-36	.px-36	.py-36	.pt-36	.pr-36	.pb-36	.pl-36
40 → 10rem	.p-40	.px-40	.py-40	.pt-40	.pr-40	.pb-40	.pl-40
44 → 11rem	.p-44	.px-44	.py-44	.pt-44	.pr-44	.pb-44	.pl-44
48 → 12rem	.p-48	.px-48	.py-48	.pt-48	.pr-48	.pb-48	.pl-48
52 → 13rem	.p-52	.px-52	.py-52	.pt-52	.pr-52	.pb-52	.pl-52
56 → 14rem	.p-56	.px-56	.py-56	.pt-56	.pr-56	.pb-56	.pl-56
60 → 15rem	.p-60	.px-60	.py-60	.pt-60	.pr-60	.pb-60	.pl-60
64 → 16rem	.p-64	.px-64	.py-64	.pt-64	.pr-64	.pb-64	.pl-64
72 → 18rem	.p-72	.px-72	.py-72	.pt-72	.pr-72	.pb-72	.pl-72
80 → 20rem	.p-80	.px-80	.py-80	.pt-80	.pr-80	.pb-80	.pl-80
96 → 24rem	.p-96	.px-96	.py-96	.pt-96	.pr-96	.pb-96	.pl-96

See the Design Tokens page for the complete spacing scale definition.

Basic usage

Add padding to all sides of an element:

```
<div class="p-8 ...">p-8</div>
```

Adding horizontal and vertical padding

Control the horizontal padding of an element with the `px-*` utilities and the vertical padding with the `py-*` utilities:

```
<div class="px-8 py-4 ...">px-8 py-4</div>
```

Padding on one side

Use `pt-*`, `pr-*`, `pb-*` and `pl-*` to add padding to one side only:

```
<div class="pt-8 pr-4 pb-2 pl-6 ...">pt-8 · pr-4 · pb-2 · pl-6</div>
```

Using logical properties

Every padding utility is a single `padding*` declaration, so they compose freely with the rest of the framework:

```
// The classes are generated from $spacing-map in _utilities.scss:
@each $key, $value in $spacing-map {
  .p-#{$key}  { padding: $value; }
  .px-#{$key} { padding-left: $value; padding-right: $value; }
  // ...
}
```

Customizing

Override `$spacing-map` before importing the utilities to change the whole scale at once:

```
@use "katanakit-css/src/scss/variables" as v with (
  $spacing-map: (
    "0": 0,
    "1": 0.5rem,
    "2": 1rem,
    "4": 2rem
  )
);
```

Width & Height

Utilities for setting the width and height of an element, generated from the `$sizing-map` for six prefixes: `w-*`, `min-w-*`, `max-w-*`, `h-*`, `min-h-*`, `max-h-*`.

Quick reference

Key	Value	`w-*`	`min-w-*`	`max-w-*`	`h-*`	`min-h-*`	`max-h-*`
`0`	`0`	`.w-0`	`.min-w-0`	`.max-w-0`	`.h-0`	`.min-h-0`	`.max-h-0`
`base`	`1px`	`.w-base`	`.min-w-base`	`.max-w-base`	`.h-base`	`.min-h-base`	`.max-h-base`
`05`	`0.125rem`	`.w-05`	`.min-w-05`	`.max-w-05`	`.h-05`	`.min-h-05`	`.max-h-05`
`1`	`0.25rem`	`.w-1`	`.min-w-1`	`.max-w-1`	`.h-1`	`.min-h-1`	`.max-h-1`
`2`	`0.5rem`	`.w-2`	`.min-w-2`	`.max-w-2`	`.h-2`	`.min-h-2`	`.max-h-2`
`3`	`0.75rem`	`.w-3`	`.min-w-3`	`.max-w-3`	`.h-3`	`.min-h-3`	`.max-h-3`
`4`	`1rem`	`.w-4`	`.min-w-4`	`.max-w-4`	`.h-4`	`.min-h-4`	`.max-h-4`
`5`	`1.25rem`	`.w-5`	`.min-w-5`	`.max-w-5`	`.h-5`	`.min-h-5`	`.max-h-5`
`6`	`1.5rem`	`.w-6`	`.min-w-6`	`.max-w-6`	`.h-6`	`.min-h-6`	`.max-h-6`
`8`	`2rem`	`.w-8`	`.min-w-8`	`.max-w-8`	`.h-8`	`.min-h-8`	`.max-h-8`
`10`	`2.5rem`	`.w-10`	`.min-w-10`	`.max-w-10`	`.h-10`	`.min-h-10`	`.max-h-10`
`12`	`3rem`	`.w-12`	`.min-w-12`	`.max-w-12`	`.h-12`	`.min-h-12`	`.max-h-12`
`16`	`4rem`	`.w-16`	`.min-w-16`	`.max-w-16`	`.h-16`	`.min-h-16`	`.max-h-16`
`20`	`5rem`	`.w-20`	`.min-w-20`	`.max-w-20`	`.h-20`	`.min-h-20`	`.max-h-20`
`24`	`6rem`	`.w-24`	`.min-w-24`	`.max-w-24`	`.h-24`	`.min-h-24`	`.max-h-24`
`32`	`8rem`	`.w-32`	`.min-`	`.max-`	`.h-32`	`.min-`	`.max-`

Key	Value	w-*	min-w-*	max-w-*	h-*	min-h-*	max-h-*
			w-32	w-32		h-32	h-32
40	10rem	.w-40	.min-w-40	.max-w-40	.h-40	.min-h-40	.max-h-40
48	12rem	.w-48	.min-w-48	.max-w-48	.h-48	.min-h-48	.max-h-48
64	16rem	.w-64	.min-w-64	.max-w-64	.h-64	.min-h-64	.max-h-64
auto	auto	.w-auto	.min-w-auto	.max-w-auto	.h-auto	.min-h-auto	.max-h-auto
full	100%	.w-full	.min-w-full	.max-w-full	.h-full	.min-h-full	.max-h-full
screen	100vw	.w-screen	.min-w-screen	.max-w-screen	.h-screen	.min-h-screen	.max-h-screen
min	min-content	.w-min	.min-w-min	.max-w-min	.h-min	.min-h-min	.max-h-min
max	max-content	.w-max	.min-w-max	.max-w-max	.h-max	.min-h-max	.max-h-max
fit	fit-content	.w-fit	.min-w-fit	.max-w-fit	.h-fit	.min-h-fit	.max-h-fit

> Note: `screen` always resolves to `100vw` — even for the height prefixes. If you need a full-height viewport box, prefer `.h-full` on a parent or your own `height: 100vh` rule.

Basic usage

```
<div class="w-64 ...">w-64</div>
<div class="w-40 h-24 ...">w-40 h-24</div>
<div class="w-24 h-24 ...">w-24 h-24</div>
```

Using percentages and content sizes

`full` is `100%`, and `min`/`max`/`fit` map to the CSS intrinsic sizes:

```
<div class="w-full ...">fills the parent width</div>
<div class="w-fit ...">shrinks to its content</div>
<div class="w-max ...">grows to its longest line</div>
```

Constraining with min and max

`max-w-*` and `min-h-*` clamp an element instead of fixing it:

```
<img class="max-w-full" src="hero.png" alt="" />
<article class="max-w-64">keeps readable line lengths</article>
```

Customizing

Override `$sizing-map` in the `utilities` module before generating:

```
@use "katanakit-css/src/scss/utilities" as u with (
  $sizing-map: (
    "0": 0,
    "1": 0.25rem,
    "4": 1rem,
    "full": 100%,
    "screen": 100vw,
    "auto": auto
  )
);

@include u.get-sizing-classes();
```

Text Color

Utilities for controlling the text color of an element, generated by `v.text-utilities()` from the color palettes. All 6 palettes with 7 shades each, plus the special colors and `:hover` variants.

Quick reference

Palette	100	200	300	400	500	600	700
neutral	`.text-neutral-100`	`.text-neutral-200`	`.text-neutral-300`	`.text-neutral-400`	`.text-neutral-500`	`.text-neutral-600`	`.text-neutral-700`
purple	`.text-purple-100`	`.text-purple-200`	`.text-purple-300`	`.text-purple-400`	`.text-purple-500`	`.text-purple-600`	`.text-purple-700`
info	`.text-info-100`	`.text-info-200`	`.text-info-300`	`.text-info-400`	`.text-info-500`	`.text-info-600`	`.text-info-700`
warning	`.text-warning-100`	`.text-warning-200`	`.text-warning-300`	`.text-warning-400`	`.text-warning-500`	`.text-warning-600`	`.text-warning-700`
danger	`.text-danger-`	`.text-danger-`	`.text-danger-`	`.text-danger-`	`.text-danger-`	`.text-danger-`	`.text-danger-`

Palette	100	200	300	400	500	600	700
	100	200	300	400	500	600	700
success	.text-success-100	.text-success-200	.text-success-300	.text-success-400	.text-success-500	.text-success-600	.text-success-700

Special	Class	Properties
black	.text-black	color: hsl(0, 0%, 3%);
white	.text-white	color: hsl(0, 0%, 98%);
transparent	.text-transparent	color: transparent;
current	.text-current	color: currentColor;

Basic usage

```
<p class="text-purple-500 ...">text-purple-500</p>
<p class="text-white ...">text-white</p>
```

Hover variants

Every text color also exists as a `:hover` variant, generated by `v.hover-utilities()`:

```
<a href="#" class="text-purple-600 hover-text-purple-400 ...">Hover me</a>
```

Variant	Palettes
.hover-text-{palette}-{100…700}:hover	.hover-text-neutral-100 … .hover-text-success-700
.hover-text-{special}:hover	.hover-text-black, .hover-text-white, .hover-text-transparent, .hover-text-current

Customizing

Color utilities are generated from `$colors` and `$special-colors` in the `variables` module:

```scss
@use "katanakit-css/src/scss/variables" as v;

// Only the purple and neutral palettes:
@include v.text-utilities(("purple", "neutral"));

// All palettes, no special colors:
@include v.text-utilities($special: false);

// Everything:
@include v.all-utilities();
```

Background Color

Utilities for controlling the background color of an element, generated by `v.bg-utilities()` from the color palettes. All 6 palettes with 7 shades each, plus the special colors and `:hover` variants.

Quick reference

Palette	100	200	300	400	500	600	700
neutral	`.bg-neutral-100`	`.bg-neutral-200`	`.bg-neutral-300`	`.bg-neutral-400`	`.bg-neutral-500`	`.bg-neutral-600`	`.bg-neutral-700`
purple	`.bg-purple-100`	`.bg-purple-200`	`.bg-purple-300`	`.bg-purple-400`	`.bg-purple-500`	`.bg-purple-600`	`.bg-purple-700`
info	`.bg-info-100`	`.bg-info-200`	`.bg-info-300`	`.bg-info-400`	`.bg-info-500`	`.bg-info-600`	`.bg-info-700`
warning	`.bg-warning-100`	`.bg-warning-200`	`.bg-warning-300`	`.bg-warning-400`	`.bg-warning-500`	`.bg-warning-600`	`.bg-warning-700`
danger	`.bg-danger-100`	`.bg-danger-200`	`.bg-danger-300`	`.bg-danger-400`	`.bg-danger-500`	`.bg-danger-600`	`.bg-danger-700`
success	`.bg-success-100`	`.bg-success-200`	`.bg-success-300`	`.bg-success-400`	`.bg-success-500`	`.bg-success-600`	`.bg-success-700`

Special	Class	Properties
black	`.bg-black`	`background-color: hsl(0, 0%, 3%);`
white	`.bg-white`	`background-color: hsl(0, 0%, 98%);`
transparent	`.bg-transparent`	`background-color: transparent;`
current	`.bg-current`	`background-color: currentColor;`

Basic usage

```
<div class="bg-purple-500 text-white ...">purple-500</div>
```

The complete swatch set of every palette is rendered on the Colors page.

Hover variants

Every background color also exists as a `:hover` variant, generated by `v.hover-utilities()`:

```
<button class="bg-purple-300 text-purple-700 hover-bg-purple-500 ...">Hover me</button>
```

Variant	Palettes
`.hover-bg-{palette}-{100…700}:hover`	`.hover-bg-neutral-100` … `.hover-bg-success-700`
`.hover-bg-{special}:hover`	`.hover-bg-black`, `.hover-bg-white`, `.hover-bg-transparent`, `.hover-bg-current`

Customizing

```
@use "katanakit-css/src/scss/variables" as v;

// Only the info palette:
@include v.bg-utilities("info");

// All palettes, no special colors:
@include v.bg-utilities($special: false);

// Everything:
@include v.all-utilities();
```

Border Radius

Utilities for controlling the border radius of an element, generated from the `$radius-map`. The base `.rounded` class (`0.25rem`) is emitted automatically with the `utilities` module.

Quick reference

Class	Properties
`.rounded`	`border-radius: 0.25rem;`
`.rounded-none`	`border-radius: 0;`
`.rounded-sm`	`border-radius: 0.125rem;`
`.rounded-md`	`border-radius: 0.375rem;`
`.rounded-lg`	`border-radius: 0.5rem;`
`.rounded-xl`	`border-radius: 0.75rem;`
`.rounded-2xl`	`border-radius: 1rem;`
`.rounded-3xl`	`border-radius: 1.5rem;`
`.rounded-full`	`border-radius: 9999px;`

Basic usage

```
<div class="rounded-lg ...">rounded-lg</div>
<div class="rounded-full ...">rounded-full</div>
```

Making circles

`rounded-full` with equal width and height produces a circle:

```
<div class="w-24 h-24 rounded-full bg-purple-500"></div>
```

Customizing

```
@use "katanakit-css/src/scss/mixins" as m with (
  $radius-map: (
    "none": 0,
    "md": 0.375rem,
    "full": 9999px
  )
);
@use "katanakit-css/src/scss/utilities" as u;

@include u.generate-effects-utilities();
```

Flex Wrap

Utilities for controlling whether flex items wrap onto multiple lines. Requires a flex container (`.flex` or `.inline-flex`).

Quick reference

Class	Properties
`.flex-wrap`	`flex-wrap: wrap;`
`.flex-nowrap`	`flex-wrap: nowrap;`

Basic usage

`flex-wrap` lets items break onto new lines when the container runs out of space; `flex-nowrap` forces a single line:

```
<div class="flex flex-wrap gap-2 w-64">…</div>
```

Preventing wrapping

```
<div class="flex flex-nowrap">
  <span class="whitespace-nowrap">Never breaks</span>
  <span class="whitespace-nowrap">Stays on one line</span>
</div>
```

Customizing

```
@use "katanakit-css/src/scss/mixins" as m with (
  $flex-wrap-list: (wrap, nowrap)
);
@use "katanakit-css/src/scss/utilities" as u;

@include u.generate-flex-utilities();
```

Font Weight

Utilities for controlling the font weight of an element, generated from the `$font-weight-values` map.

Quick reference

Class	Properties
`.font-thin`	`font-weight: 100;`
`.font-extralight`	`font-weight: 200;`
`.font-light`	`font-weight: 300;`

Class	Properties
`.font-normal`	`font-weight: 400;`
`.font-medium`	`font-weight: 500;`
`.font-semibold`	`font-weight: 600;`
`.font-bold`	`font-weight: 700;`
`.font-extrabold`	`font-weight: 800;`
`.font-black`	`font-weight: 900;`

Basic usage

```
<p class="font-thin ...">The quick brown fox — font-thin</p>
<!-- … every weight from 100 to 900 -->
```

Note: the rendered weight depends on the font actually loaded — a variable font or a family with all nine faces is required to see every step.

Customizing

```
@use "katanakit-css/src/scss/mixins" as m with (
  $font-weight-values: (
    "normal": 400,
    "medium": 500,
    "bold": 700
  )
);
@use "katanakit-css/src/scss/utilities" as u;

@include u.generate-layout-utilities();
```

Margin

Utilities for controlling an element's margin. Every value of `$spacing-map` is emitted for the seven prefixes: `m-*` (all sides), `mx-*`/`my-*` (horizontal/vertical) and `mt-*`/`mr-*`/`mb-*`/`ml-*` (one side).

Quick reference

Scale	`m-*`	`mx-*`	`my-*`	`mt-*`	`mr-*`	`mb-*`	`ml-*`
`0` → `0`	`.m-0`	`.mx-0`	`.my-0`	`.mt-0`	`.mr-0`	`.mb-0`	`.ml-0`
`05` → `0.125rem`	`.m-05`	`.mx-05`	`.my-05`	`.mt-05`	`.mr-05`	`.mb-05`	`.ml-05`

Scale	m-*	mx-*	my-*	mt-*	mr-*	mb-*	ml-*
base → 1px	.m-base	.mx-base	.my-base	.mt-base	.mr-base	.mb-base	.ml-base
1 → 0.25rem	.m-1	.mx-1	.my-1	.mt-1	.mr-1	.mb-1	.ml-1
1-5 → 0.375rem	.m-1-5	.mx-1-5	.my-1-5	.mt-1-5	.mr-1-5	.mb-1-5	.ml-1-5
2 → 0.5rem	.m-2	.mx-2	.my-2	.mt-2	.mr-2	.mb-2	.ml-2
2-5 → 0.625rem	.m-2-5	.mx-2-5	.my-2-5	.mt-2-5	.mr-2-5	.mb-2-5	.ml-2-5
3 → 0.75rem	.m-3	.mx-3	.my-3	.mt-3	.mr-3	.mb-3	.ml-3
3-5 → 0.875rem	.m-3-5	.mx-3-5	.my-3-5	.mt-3-5	.mr-3-5	.mb-3-5	.ml-3-5
4 → 1rem	.m-4	.mx-4	.my-4	.mt-4	.mr-4	.mb-4	.ml-4
5 → 1.25rem	.m-5	.mx-5	.my-5	.mt-5	.mr-5	.mb-5	.ml-5
6 → 1.5rem	.m-6	.mx-6	.my-6	.mt-6	.mr-6	.mb-6	.ml-6
7 → 1.75rem	.m-7	.mx-7	.my-7	.mt-7	.mr-7	.mb-7	.ml-7
8 → 2rem	.m-8	.mx-8	.my-8	.mt-8	.mr-8	.mb-8	.ml-8
9 → 2.25rem	.m-9	.mx-9	.my-9	.mt-9	.mr-9	.mb-9	.ml-9
10 → 2.5rem	.m-10	.mx-10	.my-10	.mt-10	.mr-10	.mb-10	.ml-10
11 → 2.75rem	.m-11	.mx-11	.my-11	.mt-11	.mr-11	.mb-11	.ml-11
12 → 3rem	.m-12	.mx-12	.my-12	.mt-12	.mr-12	.mb-12	.ml-12
14 →	.m-14	.mx-14	.my-14	.mt-14	.mr-14	.mb-14	.ml-14

Scale	m-*	mx-*	my-*	mt-*	mr-*	mb-*	ml-*
3.5rem							
16 → 4rem	.m-16	.mx-16	.my-16	.mt-16	.mr-16	.mb-16	.ml-16
20 → 5rem	.m-20	.mx-20	.my-20	.mt-20	.mr-20	.mb-20	.ml-20
24 → 6rem	.m-24	.mx-24	.my-24	.mt-24	.mr-24	.mb-24	.ml-24
28 → 7rem	.m-28	.mx-28	.my-28	.mt-28	.mr-28	.mb-28	.ml-28
32 → 8rem	.m-32	.mx-32	.my-32	.mt-32	.mr-32	.mb-32	.ml-32
36 → 9rem	.m-36	.mx-36	.my-36	.mt-36	.mr-36	.mb-36	.ml-36
40 → 10rem	.m-40	.mx-40	.my-40	.mt-40	.mr-40	.mb-40	.ml-40
44 → 11rem	.m-44	.mx-44	.my-44	.mt-44	.mr-44	.mb-44	.ml-44
48 → 12rem	.m-48	.mx-48	.my-48	.mt-48	.mr-48	.mb-48	.ml-48
52 → 13rem	.m-52	.mx-52	.my-52	.mt-52	.mr-52	.mb-52	.ml-52
56 → 14rem	.m-56	.mx-56	.my-56	.mt-56	.mr-56	.mb-56	.ml-56
60 → 15rem	.m-60	.mx-60	.my-60	.mt-60	.mr-60	.mb-60	.ml-60
64 → 16rem	.m-64	.mx-64	.my-64	.mt-64	.mr-64	.mb-64	.ml-64
72 → 18rem	.m-72	.mx-72	.my-72	.mt-72	.mr-72	.mb-72	.ml-72
80 → 20rem	.m-80	.mx-80	.my-80	.mt-80	.mr-80	.mb-80	.ml-80
96 → 24rem	.m-96	.mx-96	.my-96	.mt-96	.mr-96	.mb-96	.ml-96

Basic usage

Add margin to all sides of an element:

```
<div class="m-8 ...">m-8</div>
```

Adding horizontal and vertical margin

Control the horizontal margin with `mx-*` and the vertical margin with `my-*`:

```
<div class="mx-12 my-4 ...">mx-12 my-4</div>
```

Margin on one side

Use `mt-*`, `mr-*`, `mb-*` and `ml-*` to add margin to one side only:

```
<div class="mt-8 mr-4 mb-2 ml-6 ...">mt-8 · mr-4 · mb-2 · ml-6</div>
```

Customizing

Override `$spacing-map` before importing the utilities to change the whole scale at once:

```
@use "katanakit-css/src/scss/variables" as v with (
  $spacing-map: (
    "0": 0,
    "1": 0.5rem,
    "2": 1rem,
    "4": 2rem
  )
);
```

Position

Utilities for controlling how an element is positioned in the document.

Quick reference

Class	Properties
`.static`	`position: static;`
`.fixed`	`position: fixed;`
`.absolute`	`position: absolute;`
`.relative`	`position: relative;`
`.sticky`	`position: sticky;`

Basic usage

`relative` creates the positioning context; `absolute` removes the element from the flow and anchors it to the nearest positioned ancestor:

```
<div class="relative ...">
  <span class="absolute" style="top: .5rem; right: .5rem">…</span>
</div>
```

The framework does not emit `top`/`right`/`bottom`/`left` utilities, so pair `absolute` with your own offsets.

Fixed positioning

`fixed` anchors an element to the viewport — it stays in place when the page scrolls:

```
<header class="fixed" style="top: 0; left: 0; right: 0">…</header>
<main class="p-8"><!-- scrolls under the header --></main>
```

Sticky positioning

`sticky` keeps an element in flow until it reaches the scroll edge, then pins it:

```
<nav class="sticky" style="top: 0">…</nav>
```

Use the `z-sticky` layer from Z-Index when the sticky element needs to sit above other content.

Customizing

Position classes are generated from the `$position-values` list in the `mixins` module:

```
@use "katanakit-css/src/scss/mixins" as m with (
  $position-values: (static, fixed, absolute, relative, sticky)
);
@use "katanakit-css/src/scss/utilities" as u;

@include u.generate-layout-utilities();
```

Align Items

Utilities for controlling how flex items are positioned on the cross axis. Requires a flex container (`.flex` or `.inline-flex`).

Quick reference

Class	Properties
`.items-start`	`align-items: flex-start;`
`.items-center`	`align-items: center;`
`.items-end`	`align-items: flex-end;`
`.items-stretch`	`align-items: stretch;`

Basic usage

Align items on the cross axis of a flex container. The demo container has a fixed height so the difference is visible:

```
<div class="flex items-start gap-2 h-24">…</div>
<div class="flex items-center gap-2 h-24">…</div>
<div class="flex items-end gap-2 h-24">…</div>
```

Stretching

`items-stretch` (the CSS default) makes every item fill the cross axis:

```
<div class="flex items-stretch h-24">
  <div>…</div>
  <div>…</div>
</div>
```

Customizing

```
@use "katanakit-css/src/scss/mixins" as m with (
  $align-items-map: (
    "start": flex-start,
    "center": center,
    "end": flex-end,
    "stretch": stretch
  )
);
@use "katanakit-css/src/scss/utilities" as u;

@include u.generate-flex-utilities();
```

Border Color

Utilities for controlling the border color of an element, generated by `v.border-utilities()` from the color palettes. Pair them with the Border Width utilities.

Quick reference

Palette	100	200	300	400	500	600	700
neutral	`.border-neutral-100`	`.border-neutral-200`	`.border-neutral-300`	`.border-neutral-400`	`.border-neutral-500`	`.border-neutral-600`	`.border-neutral-700`
purple	`.border-purple-100`	`.border-purple-200`	`.border-purple-300`	`.border-purple-400`	`.border-purple-500`	`.border-purple-600`	`.border-purple-700`
info	`.border-info-100`	`.border-info-200`	`.border-info-300`	`.border-info-400`	`.border-info-500`	`.border-info-600`	`.border-info-700`
warning	`.border-warning-100`	`.border-warning-200`	`.border-warning-300`	`.border-warning-400`	`.border-warning-500`	`.border-warning-600`	`.border-warning-700`
danger	`.border-danger-100`	`.border-danger-200`	`.border-danger-300`	`.border-danger-400`	`.border-danger-500`	`.border-danger-600`	`.border-danger-700`
success	`.border-success-100`	`.border-success-200`	`.border-success-300`	`.border-success-400`	`.border-success-500`	`.border-success-600`	`.border-success-700`

Special	Class	Properties
black	`.border-black`	`border-color: hsl(0, 0%, 3%);`
white	`.border-white`	`border-color: hsl(0, 0%, 98%);`
transparent	`.border-transparent`	`border-color: transparent;`
current	`.border-current`	`border-color: currentColor;`

Basic usage

```
<div class="border-2 border-purple-500 ...">purple-500</div>
```

Using currentColor

`border-current` inherits the element's text color, which keeps borders and labels in sync:

```
<span class="border border-current text-purple-500 rounded-md
p-2">badge</span>
```

Customizing

```
@use "katanakit-css/src/scss/variables" as v;

// Only the danger and success palettes:
@include v.border-utilities(("danger", "success"));

// All palettes, no special colors:
@include v.border-utilities($special: false);

// Everything:
@include v.all-utilities();
```

Box Shadow

Utilities for controlling the box shadow of an element, generated from the `$shadow-map`. The base `.shadow` class is emitted automatically with the `utilities` module.

Quick reference

Class	Properties
`.shadow`	`box-shadow: 0 1px 3px 0 rgb(0 0 0 / 0.1), 0 1px 2px -1px rgb(0 0 0 / 0.1);`
`.shadow-none`	`box-shadow: none;`
`.shadow-sm`	`box-shadow: 0 1px 2px 0 rgb(0 0 0 / 0.05);`
`.shadow-md`	`box-shadow: 0 4px 6px -1px rgb(0 0 0 / 0.1), 0 2px 4px -2px rgb(0 0 0 / 0.1);`
`.shadow-lg`	`box-shadow: 0 10px 15px -3px`

Class	Properties
	`rgb(0 0 0 / 0.1), 0 4px 6px -4px rgb(0 0 0 / 0.1);`
`.shadow-xl`	`box-shadow: 0 20px 25px -5px rgb(0 0 0 / 0.1), 0 8px 10px -6px rgb(0 0 0 / 0.1);`
`.shadow-2xl`	`box-shadow: 0 25px 50px -12px rgb(0 0 0 / 0.25);`
`.shadow-inner`	`box-shadow: inset 0 2px 4px 0 rgb(0 0 0 / 0.05);`

Basic usage

```
<div class="shadow-md ...">shadow-md</div>
<div class="shadow-inner ...">shadow-inner</div>
```

Removing shadows

`shadow-none` removes an existing shadow — useful to override a shadow set by another rule:

```
<div class="shadow-lg shadow-none ...">no shadow</div>
```

Customizing

```
@use "katanakit-css/src/scss/mixins" as m with (
  $shadow-map: (
    "none": none,
    "sm": 0 1px 2px 0 rgb(0 0 0 / 0.05),
    "lg": 0 10px 15px -3px rgb(0 0 0 / 0.1)
  )
);
@use "katanakit-css/src/scss/utilities" as u;

@include u.generate-effects-utilities();
```

Gap

Utilities for controlling the gutters between grid and flexbox items, generated from `$spacing-map`. The bare `.gap` class is `gap: 1rem`.

Quick reference

Scale	gap-*	gap-x-*	gap-y-*
0 → 0	.gap-0	.gap-x-0	.gap-y-0
05 → 0.125rem	.gap-05	.gap-x-05	.gap-y-05
base → 1px	.gap-base	.gap-x-base	.gap-y-base
1 → 0.25rem	.gap-1	.gap-x-1	.gap-y-1
1-5 → 0.375rem	.gap-1-5	.gap-x-1-5	.gap-y-1-5
2 → 0.5rem	.gap-2	.gap-x-2	.gap-y-2
2-5 → 0.625rem	.gap-2-5	.gap-x-2-5	.gap-y-2-5
3 → 0.75rem	.gap-3	.gap-x-3	.gap-y-3
3-5 → 0.875rem	.gap-3-5	.gap-x-3-5	.gap-y-3-5
4 → 1rem	.gap-4	.gap-x-4	.gap-y-4
5 → 1.25rem	.gap-5	.gap-x-5	.gap-y-5
6 → 1.5rem	.gap-6	.gap-x-6	.gap-y-6
7 → 1.75rem	.gap-7	.gap-x-7	.gap-y-7
8 → 2rem	.gap-8	.gap-x-8	.gap-y-8
9 → 2.25rem	.gap-9	.gap-x-9	.gap-y-9
10 → 2.5rem	.gap-10	.gap-x-10	.gap-y-10
11 → 2.75rem	.gap-11	.gap-x-11	.gap-y-11
12 → 3rem	.gap-12	.gap-x-12	.gap-y-12
14 → 3.5rem	.gap-14	.gap-x-14	.gap-y-14
16 → 4rem	.gap-16	.gap-x-16	.gap-y-16
20 → 5rem	.gap-20	.gap-x-20	.gap-y-20
24 → 6rem	.gap-24	.gap-x-24	.gap-y-24
28 → 7rem	.gap-28	.gap-x-28	.gap-y-28
32 → 8rem	.gap-32	.gap-x-32	.gap-y-32
36 → 9rem	.gap-36	.gap-x-36	.gap-y-36
40 → 10rem	.gap-40	.gap-x-40	.gap-y-40
44 → 11rem	.gap-44	.gap-x-44	.gap-y-44
48 → 12rem	.gap-48	.gap-x-48	.gap-y-48

Scale	gap-*	gap-x-*	gap-y-*
52 → 13rem	.gap-52	.gap-x-52	.gap-y-52
56 → 14rem	.gap-56	.gap-x-56	.gap-y-56
60 → 15rem	.gap-60	.gap-x-60	.gap-y-60
64 → 16rem	.gap-64	.gap-x-64	.gap-y-64
72 → 18rem	.gap-72	.gap-x-72	.gap-y-72
80 → 20rem	.gap-80	.gap-x-80	.gap-y-80
96 → 24rem	.gap-96	.gap-x-96	.gap-y-96

Class	Properties
.gap	gap: 1rem;

Basic usage

Add gutters between flex items:

```
<div class="flex gap-4">
  <div>A</div>
  <div>B</div>
  <div>C</div>
</div>
```

Using row and column gaps

gap-x-* sets the column gutter, gap-y-* the row gutter:

```
<div class="grid gap-x-8 gap-y-2" style="grid-template-columns:
repeat(3, 1fr)">
  <!-- 6 cells -->
</div>
```

Using the bare gap class

.gap is a shortcut for gap: 1rem (the same value as .gap-4):

```
<div class="flex gap">…</div>
```

Customizing

Override $spacing-map before importing the utilities to change the whole scale at once:

```scss
@use "katanakit-css/src/scss/variables" as v with (
  $spacing-map: (
    "0": 0,
    "1": 0.5rem,
    "2": 1rem,
    "4": 2rem
  )
);
```

Overflow

Utilities for controlling how an element handles content that does not fit in its box. Generated from the `$overflow-values` list.

Quick reference

Class	Properties
`.overflow-auto`	`overflow: auto;`
`.overflow-hidden`	`overflow: hidden;`
`.overflow-scroll`	`overflow: scroll;`
`.overflow-visible`	`overflow: visible;`

Basic usage

Clip content that exceeds a fixed height or width:

```html
<div class="w-64 h-24 overflow-hidden ...">…</div>
<div class="w-64 h-24 overflow-auto ...">…</div>
```

- `overflow-hidden` clips with no scrolling.
- `overflow-auto` shows scrollbars only when needed.
- `overflow-scroll` always reserves scrollbar space.
- `overflow-visible` (the browser default) lets content spill out.

Combining with sizing utilities

`overflow-*` pairs with the Width & Height utilities to create scrollable regions:

```html
<div class="max-h-64 overflow-auto">
  <!-- long list -->
</div>
```

Customizing

```scss
@use "katanakit-css/src/scss/mixins" as m with (
  $overflow-values: (auto, hidden, scroll, visible)
);
@use "katanakit-css/src/scss/utilities" as u;

@include u.generate-layout-utilities();
```

Text Align

Utilities for controlling the alignment of text, generated from the `$text-align-values` list.

Quick reference

Class	Properties
`.text-left`	`text-align: left;`
`.text-center`	`text-align: center;`
`.text-right`	`text-align: right;`
`.text-justify`	`text-align: justify;`

Basic usage

```html
<p class="text-left ...">…</p>
<p class="text-center ...">…</p>
<p class="text-right ...">…</p>
<p class="text-justify ...">…</p>
```

Customizing

```scss
@use "katanakit-css/src/scss/mixins" as m with (
  $text-align-values: (left, center, right)
);
@use "katanakit-css/src/scss/utilities" as u;

@include u.generate-layout-utilities();
```

Justify Content

Utilities for controlling how flex items are distributed along the main axis. Requires a flex container (`.flex` or `.inline-flex`).

Quick reference

Class	Properties
`.justify-start`	`justify-content: flex-start;`
`.justify-center`	`justify-content: center;`
`.justify-end`	`justify-content: flex-end;`
`.justify-between`	`justify-content: space-between;`
`.justify-around`	`justify-content: space-around;`
`.justify-evenly`	`justify-content: space-evenly;`

Basic usage

```
<div class="flex justify-start ...">…</div>
<div class="flex justify-center ...">…</div>
<div class="flex justify-end ...">…</div>
<div class="flex justify-between ...">…</div>
<div class="flex justify-around ...">…</div>
<div class="flex justify-evenly ...">…</div>
```

`justify-between` pushes the first item to the start and the last to the end; `justify-around` gives every item equal half-spaces; `justify-evenly` gives every gap the same size.

Customizing

```
@use "katanakit-css/src/scss/mixins" as m with (
  $justify-content-map: (
    "start": flex-start,
    "center": center,
    "end": flex-end,
    "between": space-between,
    "around": space-around,
    "evenly": space-evenly
  )
);
@use "katanakit-css/src/scss/utilities" as u;

@include u.generate-flex-utilities();
```

Opacity

Utilities for controlling the opacity of an element, generated from the `$opacity-values` map. Unlike `hidden`, an element with `opacity-0` still takes space and receives pointer events.

Quick reference

Class	Properties
`.opacity-0`	`opacity: 0;`
`.opacity-5`	`opacity: 0.05;`
`.opacity-10`	`opacity: 0.1;`
`.opacity-20`	`opacity: 0.2;`
`.opacity-25`	`opacity: 0.25;`
`.opacity-30`	`opacity: 0.3;`
`.opacity-40`	`opacity: 0.4;`
`.opacity-50`	`opacity: 0.5;`
`.opacity-60`	`opacity: 0.6;`
`.opacity-70`	`opacity: 0.7;`
`.opacity-75`	`opacity: 0.75;`
`.opacity-80`	`opacity: 0.8;`
`.opacity-90`	`opacity: 0.9;`
`.opacity-95`	`opacity: 0.95;`
`.opacity-100`	`opacity: 1;`

Basic usage

```
<div class="opacity-50 ...">50</div>
<div class="opacity-100 ...">100</div>
```

Combining with transitions

Opacity pairs with the Transitions utilities for fade effects:

```
<button
  class="hover-bg-purple-500 bg-purple-300 duration-200 ease-out"
  style="transition-property: background-color"
>
  Fades via hover-bg
</button>
```

> Note: there is no `.hover-opacity-*` variant in this framework — the hover color variants are `.hover-text-*` and `.hover-bg-*` (see Text Color and Background Color).

Customizing

```scss
@use "katanakit-css/src/scss/mixins" as m with (
  $opacity-values: (
    "0": 0,
    "25": 0.25,
    "50": 0.5,
    "75": 0.75,
    "100": 1
  )
);
@use "katanakit-css/src/scss/utilities" as u;

@include u.generate-effects-utilities();
```

White Space

Utilities for controlling how white space inside an element is handled and how long words break. Generated from `$white-space-list` and `$overflow-wrap-list`.

Quick reference

Class	Properties
`.whitespace-nowrap`	`white-space: nowrap;`
`.whitespace-pre`	`white-space: pre;`
`.whitespace-normal`	`white-space: normal;`
`.wrap-break-word`	`overflow-wrap: break-word;`

Basic usage

`whitespace-nowrap` prevents text from wrapping. Combined with an `overflow-*` utility it produces single-line truncation:

```html
<p class="whitespace-nowrap overflow-hidden ...">…</p>
<p class="whitespace-normal ...">…</p>
```

Preserving formatting

`whitespace-pre` preserves every space and line break exactly as written in the markup:

```
<pre class="whitespace-pre ...">line one
  line two (kept indent)
line three</pre>
```

Breaking long words

`wrap-break-word` lets unbreakably long words (URLs, tokens) wrap instead of overflowing:

```
<p class="wrap-break-word
w-64 ...">https://example.com/very/long/path</p>
```

Customizing

```
@use "katanakit-css/src/scss/mixins" as m with (
  $white-space-list: (nowrap, pre, normal),
  $overflow-wrap-list: (break-word,)
);
@use "katanakit-css/src/scss/utilities" as u;

@include u.generate-layout-utilities();
```

Z-Index

Utilities for controlling the stack order of an element, generated from the `$z-layers-map`. Named layers (`dropdown` … `tooltip`) keep overlapping UI consistent across a project.

Quick reference

Class	Properties
`.z-auto`	`z-index: auto;`
`.z-0`	`z-index: 0;`
`.z-10`	`z-index: 10;`
`.z-20`	`z-index: 20;`
`.z-30`	`z-index: 30;`
`.z-40`	`z-index: 40;`
`.z-50`	`z-index: 50;`

Class	Properties
`.z-dropdown`	`z-index: 1000;`
`.z-sticky`	`z-index: 1020;`
`.z-fixed`	`z-index: 1030;`
`.z-modal`	`z-index: 1040;`
`.z-popover`	`z-index: 1050;`
`.z-tooltip`	`z-index: 1060;`

Basic usage

`z-index` only applies to positioned elements — pair it with the Position utilities:

```
<div class="relative">
  <div class="absolute z-10 ...">z-10</div>
  <div class="absolute z-20 ...">z-20</div>
  <div class="absolute z-30 ...">z-30</div>
</div>
```

Using named layers

Named layers map to the semantic roles in the `$z-layers-map` — use them so modals always sit above dropdowns, tooltips above popovers, and so on:

```
<header class="sticky z-sticky">…</header>
<nav class="absolute z-dropdown">…</nav>
<div class="fixed z-modal">…</div>
<div class="absolute z-tooltip">…</div>
```

Customizing

Override the layer map in the `mixins` module:

```
@use "katanakit-css/src/scss/mixins" as m with (
  $z-layers-map: (
    "auto": auto,
    "0": 0,
    "10": 10,
    "dropdown": 100,
    "modal": 200,
    "tooltip": 300
  )
);
@use "katanakit-css/src/scss/utilities" as u;
```

```
@include u.generate-effects-utilities();
```

Transitions

Utilities for controlling the `transition-duration` and `transition-timing-function` of an element, generated from the `$transition-duration-map` and `$transition-timing-map`.

> These utilities set the duration and easing **only**. The framework does not emit a `.transition` class — declare `transition-property` yourself (e.g. `transition: color`), or use the `transition*` utilities inside `@apply`.

Quick reference

Class	Properties
`.duration-75`	`transition-duration: 75ms;`
`.duration-100`	`transition-duration: 100ms;`
`.duration-150`	`transition-duration: 150ms;`
`.duration-200`	`transition-duration: 200ms;`
`.duration-300`	`transition-duration: 300ms;`
`.duration-500`	`transition-duration: 500ms;`
`.duration-700`	`transition-duration: 700ms;`
`.duration-1000`	`transition-duration: 1000ms;`

Class	Properties
`.ease-linear`	`transition-timing-function: linear;`
`.ease-in`	`transition-timing-function: cubic-bezier(0.4, 0, 1, 1);`
`.ease-out`	`transition-timing-function: cubic-bezier(0, 0, 0.2, 1);`
`.ease-in-out`	`transition-timing-function: cubic-bezier(0.4, 0, 0.2, 1);`

Basic usage

Hover the button to see the transition in action:

```
<button
  class="hover-bg-purple-500 bg-purple-300 duration-300 ease-out"
  style="transition-property: background-color"
>
  duration-300 ease-out
</button>
```

Customizing

```
@use "katanakit-css/src/scss/mixins" as m with (
  $transition-duration-map: (
    "fast": 100ms,
    "normal": 300ms,
    "slow": 1000ms
  ),
  $transition-timing-map: (
    "linear": linear,
    "in-out": cubic-bezier(0.4, 0, 0.2, 1)
  )
);
@use "katanakit-css/src/scss/utilities" as u;

@include u.generate-effects-utilities();
```

Layout Mixins

Grid

CSS Grid layout mixins in the `mixins` module. Configurable variables: `$grid-gap-default: f.rem(10px) !default` and `$grid-columns-default: 12 !default`.

```
@use "katanakit-css/src/scss/mixins" as m;
```

Responsive columns

Mixin	Behaviour
`m.grid-responsive($min-size, $mode: fill, $max-size: 1fr, $gap: null)`	`grid-template-columns: repeat(auto-fill|auto-fit, minmax(min, max))`. `$mode` is `fill` or `fit` (error otherwise).
`m.grid-autofill($min-size, $max-size: 1fr, $gap: null)`	Convenience for `fill`.
`m.grid-autofit($min-size, $max-`	Convenience for `fit`.

Mixin	Behaviour
`size: 1fr, $gap: null)`	
`m.grid-fixed-columns($col-width, $gap: null)`	`repeat(auto-fill, <width>)` — fixed column width, no minmax.
`m.grid-breakpoint-columns($columns-map, $gap: $grid-gap-default, $align-items: stretch)`	Different column counts per breakpoint. `$columns-map` uses a `default` key (used below `sm`) plus one key per breakpoint.

```
@use "katanakit-css/src/scss/mixins" as m;

.cards { @include m.grid-autofill(240px, 1fr, 1rem); }

.dashboard {
  @include m.grid-breakpoint-columns(("default": 1, "md": 2, "xl": 4),
1rem);
}
```

Layout containers

Mixin	Behaviour
`m.grid-container($cols: 12, $rows: null, $gap: $grid-gap-default, $place-items: center, $place-content: center, $align-items: null, $justify-content: null)`	Equal `1fr` columns. Uses `place-items`/`place-content`, or `align-items`/`justify-content` when the place-* parameters are `null`.
`m.grid-center($gap: null)`	Centering shorthand (`place-items`/`place-content: center`).
`m.grid-gap($value: $grid-gap-default, $var-name: --grid-gap)`	Sets a custom property and `gap: var(--grid-gap)`.
`m.grid-areas($areas, $gap: null)`	`grid-template-areas` with the display base.
`m.grid-area($name)`	Assigns an item to a template area.
`m.subgrid($axis: both)`	`grid-template-columns/rows: subgrid` for `column`/`row`/`both`.
`m.grid-masonry($axis: block, $gap: null)`	**Experimental** masonry via `grid-template-rows: masonry` (browser support is not universal). `$axis: inline` moves the masonry axis to

Mixin	Behaviour
	columns.

```
@use "katanakit-css/src/scss/mixins" as m;

.page {
  @include m.grid-areas(
    "header header"
    "sidebar main"
    "footer footer",
    1rem
  );

  .header  { @include m.grid-area("header"); }
  .sidebar { @include m.grid-area("sidebar"); }
  .main    { @include m.grid-area("main"); }
  .footer  { @include m.grid-area("footer"); }
}
```

Grid items

Mixin	Behaviour
`m.grid-span($cols: 1, $rows: null)`	`grid-column: span N` (+ `grid-row: span N` when `$rows` is set).
`m.grid-placement($col-start: 1, $col-end: -1, $row-start: null, $row-end: null)`	`grid-column: start / end` and, when both rows are provided, `grid-row`.
`m.grid-item-center`	`place-self: center center`.
`m.grid-item-full($col: "1 / -1", $row: null)`	Spans all columns (customizable via `$col`).

```
@use "katanakit-css/src/scss/mixins" as m;

main   { @include m.grid-placement(2, 4, 1, 2); } // grid-column: 2/4; grid-row: 1/2
header { @include m.grid-item-full(); }           // grid-column: 1 / -1
.card  { @include m.grid-span(3); }
```

Stacking items

Mixin	Behaviour
`m.grid-stack($dp: grid, $parent:`	Makes the current element (or the

Mixin	Behaviour
`null, $children: ("*"), $col: 1, $row: 1)`	named `.parent`) a grid and places every `$child` selector into the same cell at `($col, $row)`, so children overlap.
`m.grid-stack-item($col: 1, $row: 1)`	Places the current element into a stack cell.

```scss
@use "katanakit-css/src/scss/mixins" as m;

.stack { @include m.grid-stack(grid, null, ("span", "div"), 2, 1); }
// .stack { display: grid }
// .stack > span, .stack > div { grid-column: 2 / 3; grid-row: 1 /
2; }
```

`grid-stack` requires unitless numbers `>= 1` and accepts `$parent` with or without a leading dot.

Dashboard example

A complete dashboard layout combining containers, spans and breakpoints:

```scss
@use "katanakit-css/src/scss/mixins" as m;
@use "katanakit-css/src/scss/variables" as v;

.dashboard {
  @include m.grid-container(12, $gap: 1rem, $place-items: stretch,
$place-content: stretch);

  .sidebar {
    @include m.grid-span(3);
  }

  .content {
    @include m.grid-span(9);
  }

  @include m.md-down {
    grid-template-columns: 1fr;

    .sidebar,
    .content {
      @include m.grid-span(1);
```

```
    }
  }
}

.cards { @include m.grid-responsive(280px, fill, 1fr, 1.5rem); }

.pinned-card { @include m.grid-placement(2, 4, 1, 2); }

.modal-overlay {
  @include m.grid-center(0);

  .modal {
    @include m.grid-item-center;
    max-width: v.container("md");
  }
}
```

See Examples for the full `grid-dashboard.scss` source.

Flexbox

Flexbox layout mixins in the `mixins` module.

```
@use "katanakit-css/src/scss/mixins" as m;
```

Containers

Mixin	Behaviour
`m.flex-container($direction: row, $wrap: nowrap, $gap: null, $align-items: stretch, $justify-content: flex-start, $align-content: normal, $place-items: null, $place-content: null)`	Display flex with direction/wrap. `place-items` wins over `align-items`; `place-content` wins over `justify-content`/`align-content`.
`m.flex-center($gap: null)`	Centering shorthand (`place-items`/`place-content: center`).
`m.flex-gap($value: 1rem, $var-name: --flex-gap)`	Sets a custom property and `gap: var(--flex-gap)`.

```
@use "katanakit-css/src/scss/mixins" as m;

.toolbar { @include m.flex-container(row, nowrap, 0.5rem, center, space-between); }
```

```scss
.hero    { @include m.flex-center(2rem); }
.list    { @include m.flex-gap(1.5rem); }
```

Item sizing

Mixin	Behaviour
`m.flex-grow($grow: 1)`	`flex-grow`.
`m.flex-shrink($shrink: 1)`	`flex-shrink`.
`m.flex-basis($basis: auto)`	`flex-basis`.
`m.flex-item($grow: 1, $shrink: 1, $basis: auto)`	`flex: <grow> <shrink> <basis>`.
`m.flex-item-center`	`align-self: center`.
`m.flex-item-full`	`flex: 1 1 100%`.

```scss
@use "katanakit-css/src/scss/mixins" as m;

.logo   { @include m.flex-item(0, 0, auto); } // never grows, never shrinks
.search { @include m.flex-item-full(); }      // fills the remaining space
.badge  { @include m.flex-item-center; }
```

Example: a navigation bar

```scss
@use "katanakit-css/src/scss/mixins" as m;

.navbar {
  @include m.flex-container(row, nowrap, 1rem, center, space-between);

  .brand {
    @include m.flex-item(0, 0, auto);
  }

  .spacer {
    @include m.flex-item(1, 1, auto); // pushes the rest to the edges
  }

  .actions {
    @include m.flex-container(row, nowrap, 0.5rem, center, flex-end);
  }
}
```

See the Flexbox utilities for the class-based equivalents (`.flex`, `.items-*`, `.justify-*`, `.gap-*`).

Breakpoints

Technical reference of the responsive mixin engine in the `mixins` module. For a conceptual overview and the feature queries, see Breakpoints.

```scss
@use "katanakit-css/src/scss/mixins" as m;
```

Configuration

`m.$breakpoints` (all `!default`):

Tier	Width
xs	0
sm	640px
md	768px
lg	1024px
xl	1280px
2xl	1536px
3xl	1920px

```
breakpoint($from, $direction: up, $to: null)
```

Opens a `@media` query. `$from` and `$to` accept a tier name (string) or a raw px number (unitless numbers are treated as px). `bp($args...)` is an alias that forwards its arguments.

Direction	Media query
up	`(min-width: X)`
down	`(max-width: calc(X - 0.02px))`
only	`(min-width: X) and (max-width: calc(Y - 0.02px))` — requires `$to`
between	`(min-width: X) and (max-width: Y)` — requires `$to`

```scss
@include m.breakpoint("sm", up)      { /* ≥ 640px */ }
@include m.breakpoint("md", down)    { /* ≤ 767.98px */ }
@include m.breakpoint("sm", only, "md") { /* 640px–767.98px */ }
@include m.breakpoint("md", between, "xl") { /* 768px–1280px */ }
```

```
@include m.breakpoint(500px, up)      { /* raw value */ }
@include m.bp("lg", down)             { /* alias */ }
```

An invalid direction or a missing `$to` raises a compile-time `@error`.

Named mixins — `up` direction

Mixin	Equivalent
`m.xs`	`breakpoint("xs", up)`
`m.sm`	`breakpoint("sm", up)`
`m.md`	`breakpoint("md", up)`
`m.lg`	`breakpoint("lg", up)`
`m.xl`	`breakpoint("xl", up)`
`m.xxl`	`breakpoint("2xl", up)`
`m.xxxl`	`breakpoint("3xl", up)`

```
.sidebar {
  display: none;

  @include m.lg { display: block; }
}
```

Named mixins — `down` direction

Mixin	Equivalent
`m.xs-down`	`breakpoint("xs", down)`
`m.sm-down`	`breakpoint("sm", down)`
`m.md-down`	`breakpoint("md", down)`
`m.lg-down`	`breakpoint("lg", down)`
`m.xl-down`	`breakpoint("xl", down)`
`m.x2l-down`	`breakpoint("2xl", down)`
`m.x3l-down`	`breakpoint("3xl", down)`
`m.xxl-down`	alias of `m.x2l-down`
`m.xxxl-down`	alias of `m.x3l-down`

```
.grid {
  grid-template-columns: repeat(4, 1fr);

  @include m.md-down { grid-template-columns: 1fr; }
  @include m.xxl-down { padding-inline: 2rem; }
}
```

Feature query mixins

Mixin	Media query
`m.portrait`	`(orientation: portrait)`
`m.landscape`	`(orientation: landscape)`
`m.reduced-motion`	`(prefers-reduced-motion: reduce)`
`m.hoverable`	`(hover: hover) and (pointer: fine)`
`m.touch`	`(hover: none) and (pointer: coarse)`
`m.dark-mode`	`(prefers-color-scheme: dark)`
`m.light-mode`	`(prefers-color-scheme: light)`

```scss
.menu {
  @include m.touch { padding: 1rem; }   // bigger targets on touch devices
  @include m.reduced-motion { transition: none; }
}
```

Reference

Functions

Pure helper functions in the `functions` module (`src/scss/_functions.scss`). All of them raise compile-time `@error`s on invalid input instead of failing silently.

```scss
@use "katanakit-css/src/scss/functions" as f;
```

`$base-font-size: 16px !default` is the px baseline used by `rem()`/`px()`.

Unit conversion

Signature	Behaviour
`f.rem($size)`	Converts px or unitless numbers to rem using `$base-font-size`. `f.rem(16px)` and `f.rem(16)` both return `1rem`.
`f.px($size)`	Converts rem or unitless numbers to px. `f.px(1)` returns `16px`,

Signature	Behaviour
	`f.px(1.5rem)` returns 24px.
`f.to-unit($value, $unit: "rem")`	Appends a unit (`rem`, px, `em` or `%`) to a unitless number. Values with a unit are returned unchanged. Throws `@error` on an invalid unit.
`f.strip-unit($value)`	Removes the unit: `f.strip-unit(16px)` returns 16.

```scss
@use "katanakit-css/src/scss/functions" as f;

.card {
  padding: f.rem(24);       // 1.5rem
  margin: f.px(1);          // 16px
  width: f.to-unit(50, "%"); // 50%
}
```

Fluid typography

Signature	Behaviour
`f.fluid($min, $max, $min-vw: 320px, $max-vw: 1920px)`	Returns a `clamp()` expression that interpolates `$min` → `$max` across `$min-vw` → `$max-vw`. Unitless inputs are treated as px.

```scss
font-size: f.fluid(16px, 24px);
// clamp(16px, 0.5vw + 14.4px, 24px)

font-size: f.fluid(1rem, 2rem, 768px, 1280px);
// clamp(1rem, 0.1953125vw - 0.5px, 2rem)

h1 { font-size: f.fluid(2rem, 4rem); }
```

Color helpers

Signature	Behaviour
`f.tint($color, $amount: 10%)`	Mixes `$color` with white.
`f.shade($color, $amount: 10%)`	Mixes `$color` with black.
`f.saturate-color($color, $amount: 10%)`	Increases saturation via `color.scale`.
`f.desaturate-color($color, $amount: 10%)`	Decreases saturation via `color.scale`.

Signature	Behaviour
`f.complement($color)`	Returns the hue complement.
`f.contrast($color, $light: white, $dark: black)`	Chooses a readable text color from the perceived brightness of `$color`: bright colors return `$dark`, dark colors return `$light`.
`f.color-mix-var($var-name, $amount: 10%, $mix-with: white)`	Emits `color-mix(in srgb, var(--name) <pct>, <mix-with>)`. Accepts a bare variable name (`"--info-500"`) or a full `var(...)` expression.
`f.to-class($key)`	Escapes a map key for use in a class selector: `.` → `\.`, `/` → `\/`, `%` → `\%`.

```scss
@use "katanakit-css/src/scss/functions" as f;

.card {
  background: f.tint(#7d2bd4, 20%);
  border-color: f.shade(#7d2bd4, 15%);
  color: f.contrast(#7d2bd4); // white — the color is dark
}

.alert {
  background: f.color-mix-var("--danger-500", 10%);
  // background: color-mix(in srgb, var(--danger-500) 90%, white);
}

.surface {
  color: f.desaturate-color(#066bf9, 30%);
}
```

All color functions throw an `@error` when they receive a non-color.

API Reference

Complete reference for `katanakit-css` **0.1.0** (unreleased). Every signature, default and map key in this document was checked against the source under `src/scss/`. This file is documentation only — it never changes the code.

Module namespaces

All modules are consumed with lowercase `@use` and an explicit namespace:

```scss
@use "katanakit-css/src/scss/functions"   as f;
@use "katanakit-css/src/scss/variables"   as v;
@use "katanakit-css/src/scss/mixins"      as m;
@use "katanakit-css/src/scss/utilities"   as u;
```

When the modules are part of your own source tree the paths become relative, e.g. `@use "functions" as f;`. The public entry `src/scss/main.scss` loads `reset`, `variables`, `functions`, `mixins` and `utilities`, then calls:

```scss
@include v.generate-css-tokens();
@include v.generate-css-vars();
@include v.all-utilities();
@include u.generate-all-utilities();
```

Functions (`as f`)

Source: `_functions.scss`.

Unit conversion

Signature	Behaviour
`rem($size)`	Converts px or unitless numbers to rem using `$base-font-size` (default `16px`). `rem(16px)` and `rem(16)` both return `1rem`.
`px($size)`	Converts rem or unitless numbers to px. `px(1)` returns `16px`, `px(1.5rem)` returns `24px`.
`to-unit($value, $unit: "rem")`	Appends a unit (`rem`, `px`, `em` or `%`) to an already unitless number. Values that carry a unit are returned unchanged. Throws `@error` on an invalid unit.
`strip-unit($value)`	Removes the unit, e.g. `strip-unit(16px)` returns `16`.

`$base-font-size: 16px !default` is the browser default. Only override it if your project changes the root font size (it is a *px* baseline, not a 10px "rem trick" baseline).

Fluid typography

Signature	Behaviour
`fluid($min, $max, $min-vw: 320px, $max-vw: 1920px)`	Returns a `clamp()` expression that interpolates `$min` → `$max` across `$min-vw` → `$max-vw`. Unitless inputs are treated as px.

```scss
font-size: f.fluid(16px, 24px);
// clamp(16px, 0.5vw + 14.4px, 24px)

font-size: f.fluid(1rem, 2rem, 768px, 1280px);
// clamp(1rem, 0.1953125vw - 0.5px, 2rem)
```

Color helpers

Signature	Behaviour
`tint($color, $amount: 10%)`	Mixes `$color` with white.
`shade($color, $amount: 10%)`	Mixes `$color` with black.
`saturate-color($color, $amount: 10%)`	Increases saturation via `color.scale`.
`desaturate-color($color, $amount: 10%)`	Decreases saturation via `color.scale`.
`complement($color)`	Returns the hue complement.
`contrast($color, $light: white, $dark: black)`	Chooses a readable text color from the perceived brightness of `$color`: bright colors return `$dark`, dark colors return `$light`.
`color-mix-var($var-name, $amount: 10%, $mix-with: white)`	Emits `color-mix(in srgb, var(--name) <pct>, <mix-with>)`. Accepts a bare variable name (`"--info-500"`) or a full `var(...)` expression.
`to-class($key)`	Escapes a map key for use in a class selector. `.` becomes `\.`, `/` becomes `\/`, `%` becomes `\%`. `to-class("1-5")` returns the string `1-5` (the dots/hyphens of the *keys* themselves are used literally by

Signature	Behaviour
	the generators).

All color functions that receive a non-color throw an `@error`.

Variables / tokens (`as v`)

Source: `_variables.scss`.

Token maps (all `!default`)

`$font-families`

Key	Value
`sans-serif`	`ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, "Helvetica Neue", Arial, sans-serif`
`serif`	`"Times New Roman", Trebuchet, Geneva, serif`
`mono`	`monospace`

`$shadow-sizes`

Key	Value
`default`	`0 1px 3px 0 rgb(0 0 0 / 0.1), 0 1px 2px -1px rgb(0 0 0 / 0.1)`
`sm`	`0 1px 2px 0 rgb(0 0 0 / 0.05)`
`md`	`0 4px 6px -1px rgb(0 0 0 / 0.1), 0 2px 4px -2px rgb(0 0 0 / 0.1)`
`lg`	`0 10px 15px -3px rgb(0 0 0 / 0.1), 0 4px 6px -4px rgb(0 0 0 / 0.1)`
`xl`	`0 20px 25px -5px rgb(0 0 0 / 0.1), 0 8px 10px -6px rgb(0 0 0 / 0.1)`
`2xl`	`0 25px 50px -12px rgb(0 0 0 / 0.25)`
`inner`	`inset 0 2px 4px 0 rgb(0 0 0 / 0.05)`

`$container-sizes` — `sm` 24rem · `md` 28rem · `lg` 32rem · `xl` 36rem · `2xl` 42rem · `3xl` 48rem · `4xl` 56rem · `5xl` 64rem · `6xl` 72rem.

`$breakpoint-sizes` — `xs` 0 · `sm` 640px · `md` 768px · `lg` 1024px · `xl` 1280px · `2xl` 1536px · `3xl` 1920px.

`$spacing-scale`

`0` 0 · `1` 0.25rem · `2` 0.5rem · `3` 0.75rem · `4` 1rem · `5` 1.25rem · `6` 1.5rem · `8` 2rem · `10` 2.5rem · `12` 3rem · `16` 4rem · `20` 5rem · `24` 6rem · `32` 8rem.

`$radius`

`default` 0.25rem · `sm` 0.125rem · `md` 0.375rem · `lg` 0.5rem · `xl` 0.75rem · `2xl` 1rem · `3xl` 1.5rem · `full` 9999px.

`$z-layers`

`auto` auto · `0` 0 · `10` 10 · `20` 20 · `30` 30 · `40` 40 · `50` 50 · `dropdown` 1000 · `sticky` 1020 · `fixed` 1030 · `modal` 1040 · `popover` 1050 · `tooltip` 1060.

`$transition-durations` — `75` 75ms · `100` 100ms · `150` 150ms · `200` 200ms · `300` 300ms · `500` 500ms · `700` 700ms · `1000` 1000ms.

`$transition-easings`

`linear` linear · `in` `cubic-bezier(0.4, 0, 1, 1)` · `out` `cubic-bezier(0, 0, 0.2, 1)` · `in-out` `cubic-bezier(0.4, 0, 0.2, 1)`.

Accessor functions

Signature	Returns	Notes
`font-family($name)`	font stack list	`sans-serif` / `serif` / `mono`.
`shadow($size: "default")`	shadow list/value	`default` … `inner`.
`container($size)`	length	`sm` … `6xl`.
`breakpoint($name)`	px length	`xs` … `3xl`.
`spacing($size)`	length	Accepts a number **or** a string key (`spacing(4)` = `spacing("4")`).
`radius($size: "default")`	length	`default` … `full`.
`z($layer)`	number/`auto`	Accepts a number or a

Signature	Returns	Notes
		string (`z(50)` = `z("50")`, plus named layers).
`duration($ms)`	duration	Accepts a number or string (`duration(200)` = `duration("200")`).
`ease($name: "in-out")`	timing function	`linear` / `in` / `out` / `in-out`.

Every accessor raises a compile-time `@error` listing the valid keys when the requested key is missing.

```scss
@mixin generate-css-tokens(...)
@mixin generate-css-tokens(
  $fonts: true, $shadows: true, $containers: true, $spacing: true,
  $radius: true, $z: true, $durations: true, $easings: true
)
```

Emits the token families as custom properties in `:root`. Pass `false` to skip a family.

```scss
:root {
  --font-sans-serif: ui-sans-serif, system-ui, …;
  --shadow-md: …;
  --container-xl: 36rem;
  --spacing-4: 1rem;
  --z-tooltip: 1060;
  --duration-200: 200ms;
  --ease-in-out: cubic-bezier(0.4, 0, 0.2, 1);
}
```

Variable names carry no brand prefix: they are exactly `--font-*`, `--shadow-*`, `--container-*`, `--spacing-*`, `--radius-*`, `--z-*`, `--duration-*` and `--ease-*`.

Colors (`as v`)

Source: `_variables.scss` (merged from the former `_colors.scss`).

Palettes

`$colors` holds six palettes with **seven shades each** (100 lightest → 700 darkest), stored as `hsla(...)` and emitted as `hsl(...)`:

Palette	100	200	300	400	500	600	700
`neutral`	0 0% 93%	0 0% 86%	0 0% 71%	0 0% 50%	0 0% 35%	0 0% 24%	0 0% 12%
`purple`	269 60% 93%	269 70% 86%	269 70% 71%	269 66% 50%	269 66% 35%	269 60% 24%	269 50% 12%
`info`	215 95% 93%	215 95% 86%	215 95% 71%	215 95% 50%	215 95% 35%	215 95% 24%	215 95% 12%
`warning`	49 100% 93%	49 100% 86%	49 100% 71%	49 100% 50%	49 100% 35%	49 90% 24%	49 70% 12%
`danger`	3 95% 93%	3 95% 86%	3 95% 71%	3 95% 50%	3 95% 35%	3 95% 24%	3 95% 12%
`success`	147 95% 93%	147 95% 86%	147 95% 71%	147 95% 50%	147 95% 35%	147 95% 24%	147 95% 12%

The cell values above are `hsl(hue, saturation, lightness)`. There are no additional palettes and no brand-specific color names are wired into the framework (see `colors-palette.md` for the personal brand palette, which is reference material only).

`$special-colors` — `black` `hsl(0, 0%, 3%)` · `white` `hsl(0, 0%, 98%)` · `transparent` `transparent` · `current` `currentColor`.

`$shades: (100, 200, 300, 400, 500, 600, 700) !default` — iteration order for tone-driven features.

Accessor functions

Signature	Returns
`get-color($name, $shade: 300, $alpha: 1)`	A palette shade (or a special color when `$name` is special). Applies alpha when `$alpha != 1`.
`get($name, $alpha: 1)`	Shorthand: `get-color($name, 500, $alpha)`.
`alpha($name, $shade: 500, $alpha: 0.5)`	Shorthand for transparent variants: `alpha("info", 300, 0.5)` → `hsla(215, 95%, 71%, 0.5)`.

```scss
color: v.get-color("neutral", 500);  // hsl(0, 0%, 35%)
color: v.get("info");                // shade 500
border: 1px solid v.alpha("warning", 400, 0.25);
```

@mixin generate-css-vars($root: ":root", $palettes: null, $include-special: true)

Emits one custom property per shade plus the special colors:

```
:root {
  --neutral-100: hsl(0, 0%, 93%);
  /* … every palette and shade … */
  --info-500: hsl(215, 95%, 35%);
  --white: hsl(0, 0%, 98%);
  --black: hsl(0, 0%, 3%);
  --transparent: transparent;
  --current: currentColor;
}
```

`$palettes` accepts:

- `null` — all palettes;
- a palette name or list of names — only those palettes (e.g. `"info"`, `(neutral, success)`);
- a **map** — emitted as-is. This is how `theme()` feeds the inverted palettes to a custom root selector.

Utility mixins

Mixin	Classes
`text-utilities($palettes: null, $special: true)`	`.text-{palette}-{shade}`, `.text-{special}` (e.g. `.text-white`, `.text-neutral-500`, `.text-info-400`) — property `color`.
`bg-utilities($palettes: null, $special: true)`	`.bg-{palette}-{shade}`, `.bg-{special}` — property `background-color`.
`border-utilities($palettes: null, $special: true)`	`.border-{palette}-{shade}`, `.border-{special}` — property `border-color`.
`hover-utilities($palettes: null, $special: true)`	`.hover-text-{palette}-{shade}:hover`, `.hover-bg-{palette}-{shade}:hover`, plus `.hover-text-{special}:hover` / `.hover-bg-{special}:hover`.
`all-utilities($palettes: null,`	Calls the four mixins above.

Mixin	Classes
`$special: true)`	

Utility classes use **literal color values**, not `var()` references. The custom properties emitted by `generate-css-vars()` are there for your own rules.

`@mixin theme($mode, $overrides: ())`

Emits the palettes under `:root[data-theme="#{$mode}"]` (no special colors). When `$mode: "dark"`, each palette is **inverted** so the lightest tone key holds the darkest color and vice versa: shade `100` ↔ `700`, `200` ↔ `600`, `300` ↔ `500`. `$overrides` is merged over the resulting map.

Theme (`as v`)

Source: `_variables.scss` (merged from the former `_theme.scss`).

Signature	Behaviour
`theme($mode: "dark", $overrides: ())`	Thin hook that delegates to `v.theme()`.

```
@include v.theme("dark"); // inverts palettes, emits :root[data-theme="dark"]
```

Breakpoints (`as m`)

Source: `_mixins.scss` (merged from the former `_breakpoints.scss`).

`$breakpoints` (all `!default`)

`xs` 0 · `sm` 640px · `md` 768px · `lg` 1024px · `xl` 1280px · `2xl` 1536px · `3xl` 1920px.

The keys containing digits (`2xl`, `3xl`) are first-class names — pass them as strings.

Generic mixin

Signature	Behaviour
`breakpoint($from, $direction: up, $to: null)`	Opens a `@media` query. Accepts a key or a raw px number.
`bp($args...)`	Alias that forwards arguments to `breakpoint()`.

Directions:

- `up` — `(min-width: X)`
- `down` — `(max-width: calc(X - 0.02px))`
- `only` — `(min-width: X) and (max-width: calc(Y - 0.02px))`, requires `$to`
- `between` — `(min-width: X) and (max-width: Y)`, requires `$to`

```scss
@use "katanakit-css/src/scss/mixins" as m;

@include m.breakpoint("lg", down) { … }
@include m.bp("md", up) { … } // the alias forwards to
breakpoint("md", up)
```

Named mixins

up: `xs`, `sm`, `md`, `lg`, `xl`, `xxl` (alias for the `2xl` key), `xxxl` (alias for the `3xl` key).

down: `xs-down`, `sm-down`, `md-down`, `lg-down`, `xl-down`, `2xl-down` / `3xl-down`, plus the canonical aliases `xxl-down` (= `2xl-down`) and `xxxl-down` (= `3xl-down`).

```scss
@include m.xxl { … }       // @media (min-width: 1536px)
@include m.xxxl { … }      // @media (min-width: 1920px)
@include m.2xl-down { … } // @media (max-width: calc(1536px - 0.02px))
```

Feature queries

`portrait` · `landscape` · `reduced-motion` · `hoverable` (`(hover: hover) and (pointer: fine)`) · `touch` (`(hover: none) and (pointer: coarse)`) · `dark-mode` · `light-mode`.

Grid (`as m`)

Source: `_mixins.scss` (merged from the former `_grid.scss`).

Configurable variables: `$grid-gap-default: f.rem(10px) !default` and `$grid-columns-default: 12 !default`.

Responsive columns

Mixin	Behaviour
`grid-responsive($min-size, $mode: fill, $max-size: 1fr, $gap: null)`	`grid-template-columns: repeat(auto-fill|auto-fit, minmax(min, max))`. `$mode` is `fill` or

Mixin	Behaviour
	`fit` (error otherwise).
`grid-autofill($min-size, $max-size: 1fr, $gap: null)`	Convenience for `fill`.
`grid-autofit($min-size, $max-size: 1fr, $gap: null)`	Convenience for `fit`.
`grid-fixed-columns($col-width, $gap: null)`	`repeat(auto-fill, <width>)`.
`grid-breakpoint-columns($columns-map, $gap: $grid-gap-default, $align-items: stretch)`	Different column counts per breakpoint. `$columns-map` uses a `default` key (used below `sm`) and one key per breakpoint.

```scss
@use "katanakit-css/src/scss/mixins" as m;

.cards { @include m.grid-autofill(240px, 1fr, 1rem); }

.dashboard {
  @include m.grid-breakpoint-columns(("default": 1, "md": 2, "xl": 4), 1rem);
}
```

Layout containers

Mixin	Behaviour
`grid-container($cols: 12, $rows: null, $gap: $grid-gap-default, $place-items: center, $place-content: center, $align-items: null, $justify-content: null)`	Equal `1fr` columns; `place-items`/`place-content`, or `align-items`/`justify-content` when the place-* params are `null`.
`grid-center($gap: null)`	Centering shorthand (`place-items`/`place-content: center`).
`grid-gap($value: $grid-gap-default, $var-name: --grid-gap)`	Sets a custom property and `gap: var(--grid-gap)`.
`grid-areas($areas, $gap: null)`	`grid-template-areas` with a display base.
`grid-area($name)`	Assigns an item to a template area.
`grid-masonry($axis: block, $gap: null)`	**Experimental** masonry layout via `grid-template-rows: masonry` (browser support is not universal).

Mixin	Behaviour
	`$axis: inline` moves the masonry axis to columns.

Grid items

Mixin	Behaviour
`grid-span($cols: 1, $rows: null)`	`grid-column: span N` (+ `grid-row: span N`).
`grid-placement($col-start: 1, $col-end: -1, $row-start: null, $row-end: null)`	`grid-column: start / end` and, when provided, `grid-row`.
`grid-item-center`	`place-self: center center`.
`grid-item-full($col: "1 / -1", $row: null)`	Spans all columns (customize with the `$col` string).

```
main { @include m.grid-placement(2, 4, 1, 2); }  // grid-column: 2/4; grid-row: 1/2
header { @include m.grid-item-full(); }          // grid-column: 1 / -1
```

Stacking

Mixin	Behaviour
`grid-stack($dp: grid, $parent: null, $children: ("*"), $col: 1, $row: 1)`	Makes the current (or a `.parent` named) element a grid and places every `$child` selector into the same cell at `($col, $row)` so children overlap.
`grid-stack-item($col: 1, $row: 1)`	Places the current element into a stack cell.

```
.stack { @include m.grid-stack(grid, null, ("span", "div"), 2, 1); }
// .stack { display: grid } and .stack > span, .stack > div placed at 2/3 × 1/2
```

`grid-stack` requires unitless, `>= 1` numbers and accepts `$parent` with or without a leading dot.

Flex (`as m`)

Source: `_mixins.scss` (merged from the former `_flex.scss`).

Mixin	Behaviour
`flex-container($direction: row, $wrap: nowrap, $gap: null, $align-items: stretch, $justify-content: flex-start, $align-content: normal, $place-items: null, $place-content: null)`	Display flex with direction/wrap; `place-items` wins over `align-items`, `place-content` wins over `justify-content`/`align-content`.
`flex-center($gap: null)`	Centering shorthand.
`flex-gap($value: 1rem, $var-name: --flex-gap)`	Sets a custom property and `gap: var(--flex-gap)`.
`flex-grow($grow: 1)`	`flex-grow`.
`flex-shrink($shrink: 1)`	`flex-shrink`.
`flex-basis($basis: auto)`	`flex-basis`.
`flex-item($grow: 1, $shrink: 1, $basis: auto)`	`flex: <grow> <shrink> <basis>`.
`flex-item-center`	`align-self: center`.
`flex-item-full`	`flex: 1 1 100%`.

```
.toolbar { @include m.flex-container(row, nowrap, 0.5rem, center, space-between); }
.logo    { @include m.flex-item(0, 0, auto); }
```

Utilities (`as u`)

Source: `_utilities.scss`. Consolidates the former `partials/utils/` submodules (maps, spacing, sizing, flex, effects, layout, core) into a single module.

Utility token maps (all `!default`)

`sizing`, `spacing`, `gap`, `font-size` and `border-width` maps live in `_utilities.scss`. The remaining maps (`$shadow-map`, `$radius-map`, `$z-layers-map`, `$transition-duration-map`, `$transition-timing-map`, `$opacity-values`, `$font-weight-values`, `$text-align-values`, `$display-values`, `$position-values`, `$overflow-values`, `$white-space-list`, `$overflow-wrap-list`, `$flex-direction-map`, `$flex-wrap-list`, `$align-items-map`, `$justify-content-map`) live in `_mixins.scss`, where they are shared with the `@apply` registry.

`$sizing-map` — `width`/`height`/min/max values:

0 0 · base 1px · 05 0.125rem · 1 0.25rem · 2 0.5rem · 3 0.75rem · 4 1rem · 5 1.25rem · 6 1.5rem · 8 2rem · 10 2.5rem · 12 3rem · 16 4rem · 20 5rem · 24 6rem · 32 8rem · 40 10rem · 48 12rem · 64 16rem · auto auto · full 100% · screen 100vw · min min-content · max max-content · fit fit-content.

$spacing-map — padding/margin/gap values (shared by the class generator and the @apply registry):

0 0 · 05 0.125rem · base 1px · 1 0.25rem · 1-5 0.375rem · 2 0.5rem · 2-5 0.625rem · 3 0.75rem · 3-5 0.875rem · 4 1rem · 5 1.25rem · 6 1.5rem · 7 1.75rem · 8 2rem · 9 2.25rem · 10 2.5rem · 11 2.75rem · 12 3rem · 14 3.5rem · 16 4rem · 20 5rem · 24 6rem · 28 7rem · 32 8rem · 36 9rem · 40 10rem · 44 11rem · 48 12rem · 52 13rem · 56 14rem · 60 15rem · 64 16rem · 72 18rem · 80 20rem · 96 24rem.

$font-size-map — xs 0.75rem · sm 0.875rem · base 1rem · lg 1.125rem · xl 1.25rem · 2xl 1.5rem · 3xl 1.875rem · 4xl 2.25rem. Emitted automatically as .text-xs … .text-4xl.

$border-width-map — 0 0 · 2 2px · 4 4px · 8 8px. Emitted automatically as .border-0/.border-2/.border-4/.border-8; .border (1px solid) is also emitted.

$flex-direction-map — row, row-reverse, col, col-reverse. **$flex-wrap-list** — wrap, nowrap. **$align-items-map** — start (flex-start), center, end (flex-end), stretch. **$justify-content-map** — start, center, end, between (space-between), around (space-around), evenly (space-evenly).

$shadow-map (effects) — none, sm, md, lg, xl, 2xl, inner. **$radius-map** — none 0 · sm 0.125rem · md 0.375rem · lg 0.5rem · xl 0.75rem · 2xl 1rem · 3xl 1.5rem · full 9999px. **$z-layers-map** — auto, 0, 10, 20, 30, 40, 50, dropdown, sticky, fixed, modal, popover, tooltip. **$transition-duration-map** — 75, 100, 150, 200, 300, 500, 700, 1000 (ms). **$transition-timing-map** — linear, in, out, in-out. **$opacity-values** — 0, 5, 10, 20, 25, 30, 40, 50, 60, 70, 75, 80, 90, 95, 100.

$font-weight-values — thin 100 · extralight 200 · light 300 · normal 400 · medium 500 · semibold 600 · bold 700 · extrabold 800 · black 900. **$text-align-values** — left, center, right, justify.

$display-values (a map so hidden can map to none) — block, inline-block, inline, flex, inline-flex, grid, inline-grid, table, table-row, table-cell, hidden (none), contents. **$position-values** — static, fixed, absolute, relative, sticky. **$overflow-values** — auto, hidden, scroll, visible. **$white-space-list** — nowrap, pre, normal. **$overflow-wrap-list** — break-word.

Core mixins — `_mixins.scss`

Mixin	Behaviour
`vars-list($list, $prefix: null)`	Emits `--{prefix}-{value}: {value}` (or `--{value}`) for each item of a list.
`vars-map($map, $prefix: null)`	Emits `--{prefix}-{key}: {value}` (or `--{key}`) for each pair of a map.
`absolute-center($pos: absolute, $y: 50%, $x: 50%)`	`position`, `top: $y`, `left: $x`, `transform: translate(-50%, -50%)`.
`center-mx($xvalue: auto)`	`margin-inline: $xvalue`.
`center-my($yvalue: auto)`	`margin-block: $yvalue`.
`utils-classes($input, $property, $prefix: null)`	Generates `.prefix-value { property: value }` (or unprefixed) from a list or a map. The class suffix is the *key* for a map, the *value* for a list.
`utils-classes-hover($input, $property, $prefix: null)`	Same, but emits `.prefix-key:hover`.

```scss
@include m.utils-classes((auto, hidden), overflow, "overflow");
// .overflow-auto, .overflow-hidden

@include m.utils-classes((auto: auto, hidden: hidden), overflow,
"overflow");
// identical output

@include m.utils-classes((flex, grid), display);
// .flex, .grid (no prefix)
```

Class generators (opt-in, invoked by `generate-all-utilities()`)

Generator	Classes
`u.get-sizing-classes($map: u.$sizing-map)`	`.w-*`, `.min-w-*`, `.max-w-*`, `.h-*`, `.min-h-*`, `.max-h-*`.
`u.generate-flex-utilities($direction: …, $wrap: …, $align: …, $justify: …, $gap: …)`	`.flex-row` / `.flex-col` / reverses, `.flex-wrap` / `.flex-nowrap`, `.items-*`, `.justify-*`, `.gap-0` … `.gap-8`.
`u.generate-effects-utilities()`	`.shadow-*`, `.rounded-*`, `.z-*`, `.duration-*`, `.ease-*`, `.opacity-*`.
`u.generate-layout-utilities()`	display, position, `.overflow-*`, `.text-left/center/right/justify`, `.font-*`,

Generator	Classes
	`.whitespace-*`, `.wrap-break-word`.
`u.generate-all-utilities()`	Sizing + flex + effects + layout.

Spacing, text-size and border-width classes are **not** part of these generators — they are emitted automatically when the module loads (see below).

Automatic output when the module is loaded

Loading `utilities` executes the automatic rules at the top level. The compiled sheet therefore always contains:

- **Spacing** — for every key of `$spacing-map`: `.p-*`, `.px-*`, `.py-*`, `.pt-*`, `.pr-*`, `.pb-*`, `.pl-*`, `.m-*`, `.mx-*`, `.my-*`, `.mt-*`, `.mr-*`, `.mb-*`, `.ml-*`, `.gap-*`, `.gap-x-*`, `.gap-y-*`.
- **Typography** — `.text-xs` … `.text-4xl` from `$font-size-map`.
- **Border widths** — `.border-0`, `.border-2`, `.border-4`, `.border-8` from `$border-width-map`, plus `.border { border-width: 1px; border-style: solid }`.
- **Base effects** — `.gap { gap: 1rem; }`, `.shadow` (default shadow) and `.rounded { border-radius: 0.25rem; }`.

```
.px-4 { padding-left: 1rem; padding-right: 1rem; }
.my-8 { margin-top: 2rem; margin-bottom: 2rem; }
.gap-x-4 { column-gap: 1rem; }
.text-lg { font-size: 1.125rem; }
.border-2 { border-width: 2px; }
.border { border-width: 1px; border-style: solid; }
```

Everything else is opt-in via the generators above (or `main.scss`).

Notes on naming

- Keys are literal strings, kept kebab-style: `.p-05`, `.p-1-5`, `.p-2-5`, `.p-3-5`, `.p-base`, `.w-screen { width: 100vw }`, `.hidden { display: none }`, `.font-normal { font-weight: 400 }`.
- `generate-layout-utilities` emits `.hidden` because `$display-values` maps hidden → none.

The apply system (`as m`)

Source: `_mixins.scss` (merged from the former `_apply.scss`).

`@mixin register-utility($name, $styles)`

Registers a named utility. `$styles` is a map of CSS declarations.

`@mixin apply($utilities...)`

Emits every registered declaration for each utility name, in order. Throws an `@error` listing the offending name when a utility is not registered.

```
@use "katanakit-css/src/scss/mixins" as m;

@include m.register-utility("fade-in", (animation: fade-in 300ms
ease));
.card {
  @include m.apply(flex, flex-col, p-4, rounded-lg, shadow-md, bg-
white);
}
```

Automatic registry

`_mixins.scss` registers a large set of utilities when loaded. Most of the registry is derived from the **same maps** that drive the class generators (`$radius-map`, `$shadow-map`, `$z-layers-map`, `$transition-duration-map`, `$transition-timing-map`, `$opacity-values`, `$display-values`, `$position-values`, `$overflow-values`, `$text-align-values`, `$font-weight-values`, `$flex-direction-map`, `$flex-wrap-list`, `$align-items-map`, `$justify-content-map`), so the names match the map keys exactly (`p-4`, `m-2`, `gap-4`, `rounded-lg`, `shadow-md`, `z-10`, `duration-200`, `ease-out`, `opacity-50`, …). Spacing names follow the `$spacing-scale` token map from `_variables.scss` — the class set from `$spacing-map` is wider than the registered set.

Categories:

- **Display** — `block`, `inline-block`, `inline`, `flex`, `inline-flex`, `grid`, `hidden`, plus every key of `$display-values`.
- **Flexbox** — `flex-row`, `flex-col`, `flex-row-reverse`, `flex-col-reverse`, `flex-wrap`, `flex-nowrap`, `flex-1`, `flex-auto`, `flex-none`, `items-*`, `justify-*`.
- **Spacing** — for every key of `$spacing-scale` (variables): `p-*`, `px-*`, `py-*`, `pt-*`, `pb-*`, `pl-*`, `pr-*`, `m-*`, `mx-*`, `my-*`, `mt-*`, `mb-*`, `ml-*`, `mr-*`, `gap-*`, `gap-x-*`, `gap-y-*`.
- **Typography** — `text-xs` (0.75rem) … `text-4xl` (2.25rem), `text-left`, `text-center`, `text-right`, `font-thin` … `font-black`.

- **Sizing** — `w-full`, `w-screen`, `w-auto`, `h-full`, `h-screen`, `h-auto`.
- **Borders** — `rounded`, `rounded-md`, `rounded-lg`, `rounded-xl`, `rounded-full`, `border`, `border-0`, `border-2`, plus every key of `$radius-map` (`rounded-none` … `rounded-full`).
- **Effects** — `shadow`, `shadow-sm` … `shadow-2xl` (every key of `$shadow-map`), `opacity-0` … `opacity-100` (every key of `$opacity-values`), `z-*`, `duration-*`, `ease-*`.
- **Grid helpers** — `grid-cols-1`, `grid-cols-2`, `grid-cols-3`, `grid-cols-4`, `grid-cols-6`, `grid-cols-12`, `col-span-full`.
- **Position/overflow** — `static`, `fixed`, `absolute`, `relative`, `sticky`, `overflow-auto/hidden/scroll/visible`.
- **Colors** — `text-white`, `text-black`, `bg-white`, `bg-black`, `bg-transparent`.
- **Transitions** — `transition`, `transition-colors`, `transition-opacity`, `transition-transform`.
- **White space** / **wrapping** — `whitespace-nowrap/pre/normal`, `wrap-break-word`.

Important. Registering a utility for `apply` does **not** create a CSS class. Because spacing, text sizes and border widths are now generated as classes automatically, the registry-only names are limited to `flex-1`, `flex-auto`, `flex-none`, `grid-cols-1/2/3/4/6/12`, `col-span-full` and the `transition*` helpers — those exist only inside `apply()` and must never be assumed to exist as classes in the compiled CSS.

Error handling

Invalid keys, wrong argument types and unsupported directions raise compile-time `@error` messages that name the offending value and, when applicable, the valid options. There is no silent fallback.

Architecture

This document explains how `katanakit-css` is organised, how the modules relate to each other and how a source `.scss` file becomes the artifacts you ship.

> TL;DR — `katanakit-css` is **build-time SCSS**. There is no JavaScript runtime. Everything is produced by the Sass compiler from `!default` token maps.

---

1. Overview

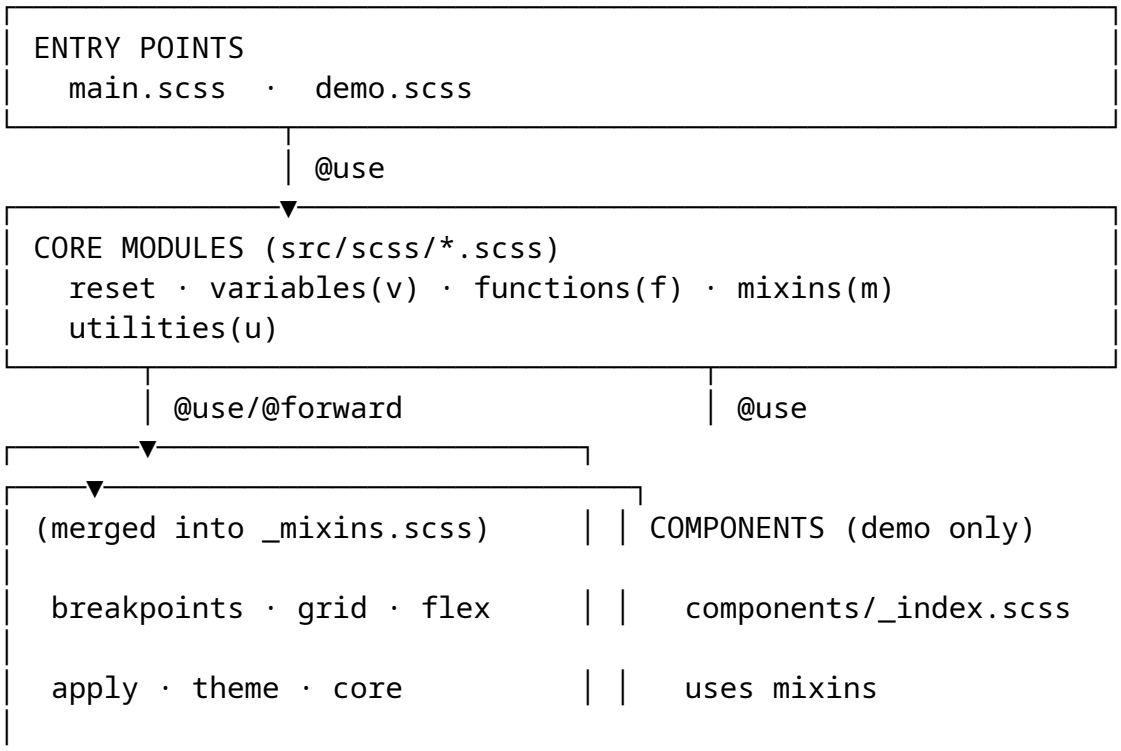
The framework is a collection of Sass **modules** (partials) plus two compilable entries:

Entry	What it emits	Used for
`src/scss/main.scss`	Public "full sheet": reset + token custom properties + color variables and utilities + all utility generators. **No components.**	npm artifact `dist/css/katanakit.css` and `@use "katanakit-css/src/scss/main"` consumers.
`src/scss/demo.scss`	`@use "main"` + `@use "components/index"` (example components built on `apply`).	Local demo only (Vite).

Components intentionally stay out of `main.scss`: they are *example* styles, not framework API.

---

2. Layer model



└──────────────────────────────────────┘
└──┘

Modules never `@import` each other; they use modern `@use` (lowercase) with explicit namespaces. The former `partials/` and `partials/utils/` layers have been consolidated: `_variables.scss` absorbs colors and theme, `_mixins.scss` absorbs breakpoints, grid, flex, apply and core generators, and `_utilities.scss` absorbs all maps and class generators.

---

3. Responsibilities of each partial

Partial	Namespace	Responsibility
`_reset.scss`	—	Modern reset. Defaults are exposed as `:root` custom properties (`--m-reset`, `--box-sizing`, `--min-h-screen`, …) so a theme can change them without editing the reset.
`_variables.scss`	`as v`	`!default` token maps (fonts, shadows, containers, breakpoints, spacing, radius, z, durations, easings), accessor functions, `generate-css-tokens()`, color tokens/accessors (`get-color`/`get`/`alpha`), `generate-css-vars()`, color utilities, and `theme()`.
`_functions.scss`	`as f`	Pure helpers: unit conversion, `fluid()` → `clamp()`, color manipulation, `color-mix-var()`, `to-class()`. No CSS output.
`_mixins.scss`	`as m`	Breakpoint queries

Partial	Namespace	Responsibility
		(`breakpoint()`/`bp()`, named aliases, feature queries), grid composition mixins, flex container/item mixins, `@apply` registry (`register-utility()`/`apply()`), core generators (`utils-classes`, `vars-list`/`vars-map`, centering helpers), and theme hook.
`_utilities.scss`	`as u`	Utility token maps (`$sizing-map`, `$font-size-map`, `$border-width-map`, etc.) plus all class generators (`get-sizing-classes`, `generate-flex-utilities`, `generate-effects-utilities`, `generate-layout-utilities`, `generate-all-utilities`).

4. The utility engine

The utility system is deliberately map-driven so that tokens and generated classes cannot drift apart.

4.1 Maps are the single source of truth

`_utilities.scss`, `_variables.scss` and `_mixins.scss` hold the utility token maps. The sizing/font-size/border-width maps live in `_utilities.scss`; `$spacing-map` lives in `_variables.scss` (shared by the class generator and the `@apply` registry); the rest live in `_mixins.scss` next to the generic class generator:

- `$sizing-map` (width/height values, incl. `full`, `screen`, `min`, `max`, `fit`),

- `$spacing-map` (padding/margin/gap scale; keys are literal: `"05"`, `"1-5"`, `"2-5"`, `"3-5"`, `"base"` …),
- `$font-size-map` (`.text-xs` … `.text-4xl`), `$border-width-map` (`.border-0/2/4/8`),
- `$flex-direction-map`, `$flex-wrap-list`, `$align-items-map`, `$justify-content-map`,
- `$shadow-map`, `$radius-map`, `$z-layers-map`, `$transition-duration-map`, `$transition-timing-map`, `$opacity-values`,
- `$font-weight-values`, `$text-align-values`, `$display-values`, `$position-values`, `$overflow-values`, `$white-space-list`, `$overflow-wrap-list`.

All maps are `!default`, so downstream projects can override them with the Sass `with` clause.

4.2 From maps to CSS classes

```
_utilities.scss (maps) ──► generators (_utilities.scss)
                                  │  use m.utils-classes($input,
$property, $prefix)
                                  ▼
                      .w-full { width: 100% }
                      .hidden { display: none }   // $display-values
maps hidden → none
                      .font-normal { font-weight: 400 }
                      .gap-4 { gap: 1rem }
                      …
```

- `_mixins.scss` provides the generic `utils-classes()` / `utils-classes-hover()` mixins plus CSS-variable helpers (`vars-list`, `vars-map`) and centering helpers (`absolute-center`, `center-mx`, `center-my`).
- `_utilities.scss` emits, automatically at load time: padding and margin classes in every direction (`.p-*`, `.px-*`, `.py-*`, `.pt-*`, …, `.m-*`, `.mx-*`, `.my-*`, …), `gap-*`/`gap-x-*`/`gap-y-*` for every `$spacing-map` key, text sizes (`.text-xs` … `.text-4xl`), border widths (`.border`, `.border-0/2/4/8`), plus `.gap`, `.shadow` and `.rounded`. This means loading the module is enough to get the common spacing and typography classes.
- The rest is **opt-in**: `get-sizing-classes()`, `generate-flex-utilities()`, `generate-effects-utilities()` and `generate-layout-utilities()`. `generate-all-utilities()` calls them all, which `main.scss` invokes.

4.3 The apply registry mirrors the same maps

`_mixins.scss` builds its automatic registry with `@each` loops **over the same utility maps that drive the class generators** — `$radius-map`, `$shadow-map`, `$z-layers-map`, `$transition-duration-map`, `$transition-timing-map`, `$opacity-values`, `$display-values`, `$position-values`, `$overflow-values`, `$text-align-values`, `$font-weight-values`, `$flex-direction-map`, `$flex-wrap-list`, `$align-items-map`, `$justify-content-map` — plus the `$spacing-scale` token map from `_variables.scss` for spacing names**. That keeps `apply(flex, p-4, rounded-lg, …)` consistent with the generated `.flex`, `.p-4` and `.rounded-lg` classes, and means a scale change propagates to both systems at once.

The registry also carries hand-written helpers that are *not* generated as classes (`flex-1`, `flex-auto`, `flex-none`, grid columns (`grid-cols-*`, `col-span-full`), transitions, …). Those names are only available inside `@include a.apply(...)`.

4.4 Color utilities

Color utilities do **not** go through the generic engine. `_variables.scss` emits them with a dedicated helper that iterates `$colors` × `$shades` and `$special-colors`:

```
.text-neutral-500  { color: hsl(0 0% 35%) }
.bg-info-300       { background-color: … }
.border-white      { border-color: … }
.hover-bg-success-600:hover { … }
```

The palette custom properties (`--neutral-500`, `--white`, …) are a parallel output of `generate-css-vars()`; the utility classes keep **literal values** so class-based CSS purging can remove them safely while the token variables stay in `:root`.

---

5. Tokens and the `:root` block

`main.scss` calls, in order:

```scss
@use "./reset";            // reset emits its own :root defaults
@use "./variables" as v;
@use "./functions" as f;
@use "./mixins" as m;
@use "./utilities" as u;

@include v.generate-css-tokens();  // --font-*, --shadow-*, --spacing-*, …
```

```scss
@include v.generate-css-vars();     // --neutral-100…, --white, …
@include v.all-utilities();         // .text-*, .bg-*, .border-*, hover
@include u.generate-all-utilities();
```

The CSS custom properties are grouped under `:root`. Because every token map is `!default`, a consumer can reconfigure the whole output by loading the `variables` module with a `with` clause *before* anything else loads it (a Sass module can only be configured once per compilation).

Dark themes do not edit `:root`: `theme("dark")` emits an *additional* block under `:root[data-theme="dark"]` with each palette inverted (100 ↔ 700, 200 ↔ 600, 300 ↔ 500), leaving the light defaults untouched.

---

6. Build pipeline

Two independent pipelines produce the artifacts:

```
(1) npm artifact
    src/scss/main.scss
          │  yarn build:css
          │  sass src/scss/main.scss dist/css/katanakit.css
          │  --no-source-map --style=compressed
          ▼
    dist/css/katanakit.css            ◀── shipped, referenced by
"style"

(2) local demo
    src/scss/demo.scss  +  index.html + demo/main.js
          │  yarn dev         →  vite dev server (HMR, port 4321)
          │  yarn build:demo  →  vite build
          │                      PostCSS autoprefixer  +  PurgeCSS
          ▼
    demo-dist/ (index.html + assets/*.css + assets/*.js)
```

Notes on the demo build (`vite.config.js`):

- `publicDir: false` — there is no static folder; the CSS is compiled from the SCSS module graph.
- PurgeCSS content is `index.html` + `demo/**/*.{js,ts}`. A safelist keeps variant-looking tokens (`hover:*`, `md:*`, …) and keyframes/font-faces are preserved.

- `variables: false` — PurgeCSS keeps the custom properties in `:root` (the token/color variables) even though utility classes use literal values.
- Output goes to `demo-dist/`, intentionally **not** `dist/`, so the demo never overwrites the npm CSS artifact.
- The `dist/css/katanakit.css` artifact is produced by the Sass CLI alone; run your own PostCSS (e.g. autoprefixer) when you consume the SCSS source if you need prefixing for older browsers.

The test suite (`vitest`, `test/**/*.test.mjs`) compiles the entries and fixtures in memory via the Sass JS API (`loadPaths: src/scss`) and asserts on the compiled CSS — it never writes to `src/` or `dist/`.

---

7. Conventions that keep the architecture intact

1. **One responsibility per partial** — put new capabilities in the module that owns that concern (tokens and colors in `_variables.scss`, mixins in `_mixins.scss`, utility maps and generators in `_utilities.scss`, …).
2. **Modern module syntax only** — `@use`/`@forward`, never `@import`.
3. **Maps are `!default`** — any scale can be overridden without forks.
4. **Kebab-case literal keys** — utility keys (`.p-05`, `.p-1-5`, `.w-screen`) match the map keys exactly; no bracket conversion layer.
5. **Generated classes vs registry** — only mixins explicitly emit classes; registering a utility for `apply` never creates a class by itself.
6. **`main.scss` stays component-free** — example components live in `components/_index.scss` and are pulled in by `demo.scss`, never by the public sheet.

Examples

Four working examples live in the `examples/` directory of the repository. Each one is a standalone SCSS file you can compile with:

```
sass --load-path src/scss examples/<category>/<file>.scss out.css
```

1. Custom design tokens

`examples/tokens/custom-tokens.scss` — override the `!default` token maps with a `with` clause **before** anything uses them, then consume the new values through the accessors.

```scss
// ============================================================
//  examples/tokens/custom-tokens.scss
//  Cómo sobreescribir los tokens del framework ANTES de usarlos.
// ============================================================

// 1) Override de mapas con @use ... with (antes de cualquier
@include)
@use "variables" as v with (
  $spacing-scale: (
    "0": 0,
    "1": 0.5rem,
    "2": 1rem,
    "3": 1.5rem,
    "4": 2rem
  ),
  $radius: (
    "default": 0.5rem,
    "full": 9999px
  )
);

// 2) Emite los tokens como custom properties en :root
@include v.generate-css-tokens();

// 3) Usa las funciones de acceso con los nuevos valores
.hero {
  padding: v.spacing(4);
  border-radius: v.radius("default");
  box-shadow: v.shadow("lg");
  font-family: v.font-family("sans-serif");
}
```

The key idea: **configure once, before first use**. Because a Sass module can only be configured once per compilation, the `with` clause must come before any other module loads `variables`. See Design Tokens for the full token reference.

2. Grid dashboard layout

`examples/layout/grid-dashboard.scss` — a dashboard built with the grid mixins: a 12-column container, explicit spans, a mobile-first collapse via `md-down` and a responsive card grid with `auto-fill`.

```scss
// ============================================================
//  examples/layout/grid-dashboard.scss
//  Layout de dashboard con los mixins de grid y breakpoints.
// ============================================================

@use "mixins" as m;
@use "variables" as v;

.dashboard {
  @include m.grid-container(12, $gap: 1rem, $place-items: stretch,
$place-content: stretch);

  // Sidebar: ocupa 3 columnas, el contenido 9
  .sidebar {
    @include m.grid-span(3);
  }

  .content {
    @include m.grid-span(9);
  }

  // En pantallas medianas o menores, todo apila a 1 columna
  @include m.md-down {
    grid-template-columns: 1fr;

    .sidebar,
    .content {
      @include m.grid-span(1);
    }
  }
}

// Rejilla responsive de tarjetas con auto-fill
.cards {
  @include m.grid-responsive(280px, fill, 1fr, 1.5rem);
}

// Colocación explícita de un ítem
.pinned-card {
  @include m.grid-placement(2, 4, 1, 2);
}
```

```scss
// Centrado absoluto
.modal-overlay {
  @include m.grid-center(0);

  .modal {
    @include m.grid-item-center;
    max-width: v.container("md");
  }
}
```

The sidebar and content share the same 12-column grid; below `md` everything falls back to a single column. See the Grid mixins for every mixin used here.

3. Dark theme

`examples/theme/dark-mode.scss` — publish the inverted palettes under `:root[data-theme="dark"]` and add a system-preference hook on top.

```scss
// ============================================================
//  examples/theme/dark-mode.scss
//  Tema oscuro automático: las paletas se invierten (tono 100
//  ↔ 700) bajo :root[data-theme='dark'].
// ============================================================

@use "variables" as v;
@use "mixins" as m;

// Emite :root[data-theme='dark'] con las paletas invertidas
@include v.theme("dark");

// Y añade tu propio hook de sistema si quieres:
@include m.dark-mode {
  :root {
    color-scheme: dark;
  }
}
```

Then toggle the attribute from markup or JavaScript:

```html
<html data-theme="dark">…</html>
```

Read Dark Mode for the inversion table and JavaScript snippets.

4. Button and card components

`examples/components/button-card.scss` — composite components built with the `@apply` system.

```scss
// ============================================================
//  examples/components/button-card.scss
//  Componentes compuestos con el sistema @apply propio.
// ============================================================

@use "mixins" as m;

.card {
  @include m.apply(flex, flex-col, p-4, rounded-lg, shadow-md, bg-
white);

  &:hover {
    @include m.apply(shadow-lg);
  }
}

.button {
  @include m.apply(
    inline-flex,
    items-center,
    justify-center,
    px-4,
    py-2,
    rounded-md,
    font-semibold,
    transition-colors
  );

  &:hover {
    @include m.apply(bg-white, text-black);
  }
}
```

Each component stays a single, readable block of utility names — the registry expands them to plain declarations at compile time, so the output CSS contains no utility classes for these components.

Roadmap

A living list of where `katanakit-css` (SCSS mini-framework) is heading. Contributions are welcome — pick an item, discuss it in an issue and open a pull request.

Legend: `[x]` done · `[ ]` planned (no commitment to an order).

Current status

The framework is at **0.1.0 (unreleased)**. The shipped scope is documented in CHANGELOG.md; the API in the API Reference.

Shipped for 0.1.0

- ☒ Modular SCSS architecture with modern `@use`/`@forward` partials and a component-free public entry (`src/scss/main.scss`).
- ☒ Design-token system — `!default` maps for fonts, shadows, containers, breakpoints, spacing, radius, z-index, transition durations and easings, emitted as CSS custom properties (`--font-*`, `--shadow-*`, `--spacing-*`, `--radius-*`, `--z-*`, `--duration-*`, `--ease-*`, …).
- ☒ Color system — six semantic palettes (`neutral`, `purple`, `info`, `warning`, `danger`, `success`) with **7 tones each** (100–700) plus four special colors; CSS variables, utility classes (`text`/`bg`/`border`/ hover) and dark-theme inversion via `:root[data-theme="dark"]`.
- ☒ Responsive breakpoints — 7 tiers (`xs` … `3xl`), generic `breakpoint()`/`bp()` with up/down/only/between, named `up` and `down` aliases (`xxl`/`xxxl`, `xxl-down`/`xxxl-down`) and feature queries.
- ☒ CSS Grid mixins — responsive auto-fill/fit, fixed columns, per-breakpoint columns, containers, areas, placement, stacking, item helpers, subgrid and (experimental) masonry.
- ☒ Flexbox mixins — container, centering, gap and item helpers.
- ☒ Pure functions — `rem()`, `px()`, `to-unit()`, `strip-unit()`, `fluid()` → `clamp()`, `tint()`, `shade()`, `saturate-color()`, `desaturate-color()`, `complement()`, `contrast()`, `color-mix-var()`, `to-class()`.
- ☒ Utility engine driven by `_utilities.scss` maps — padding and margin classes in every direction (`.p-*`, `.px-*`, `.py-*`, `.pt-*`, `.m-*`, `.mx-*`, `.my-*`, …), `gap-*`/`gap-x-*`/`gap-y-*`, text sizes (`.text-xs` … `.text-4xl`) and border

widths (`.border`, `.border-0/2/4/8`) active by default; sizing/flex/effects/layout generators opt-in.

- ☒ `@apply`-style registry (`register-utility` + `apply`) kept in sync with the utility maps.
- ☒ Extended Tailwind-style preflight reset whose defaults are overrideable custom properties.
- ☒ Vite demo (`index.html` + `demo/main.js`) with HMR, a **version switcher** (`demo/version-switcher.js`, `yarn build:versions` → `public/versions/`) and a PurgeCSS production build into `demo-dist/`.
- ☒ Example components built on `@apply` (`components/_index.scss`).
- ☒ Test suite (vitest, 26 tests over fixtures) and full English documentation (README + `docs/` + CONTRIBUTING/SECURITY/CHANGELOG), plus the Astro + Starlight documentation site under `site/`.

Planned

Utilities as real classes

- ☐ Extract directional `gap`/`margin`/`padding` class generation into a dedicated loop so future scales can add keys without touching the auto-emit block.

Token architecture

- ☐ **Unify the two spacing scales**: `$spacing-scale` (token variables and the `@apply` registry) and `$spacing-map` (utility classes) currently hold different key sets. Unifying them would make `--spacing-*` variables, `.p-*` classes and `apply()` names one source of truth.
- ☐ Configurable prefix for the generated CSS custom properties (an optional `$prefix` on `generate-css-tokens()`/`generate-css-vars()`).
- ☐ Evaluate a third scale tier (`3xl`/`4xl` sizes) and named opacity/weight aliases.

Colors

- ☐ Extend the palette system beyond 7 shades (e.g. 50 and 800/900 tones) without breaking existing 100–700 consumers.
- ☐ Hover variants for `border-*` (`hover-border-*`), mirroring `hover-text-*`/`hover-bg-*`.
- ☐ Documentation for the personal brand palette (`colors-palette.md`) as a start-point theme, still kept separate from the framework palettes.

Components & extras

- ☐ Ship an opt-in `components` module on npm (only the mixins/registry it needs), keeping the public sheet component-free.
- ☐ Container/`max-w` helper mixins layered on the `--container-*` tokens.

DX & tooling

- ☐ Continuous integration with GitHub Actions (test on multiple Dart Sass versions, build both artifacts, cache Yarn).
- ☐ Document the sibling TypeScript library `katanakit-js` next to this repo so the two projects are clearly distinguished.

Contributing to the roadmap

If you want to tackle a planned item, open an issue first so the approach is agreed (some items touch breaking API decisions). See CONTRIBUTING.md for the development workflow.